

The Hypervolume Indicator: Computational Problems and Algorithms

ANDREIA P. GUERREIRO, University of Coimbra, Portugal and INESC-ID, Portugal

CARLOS M. FONSECA, University of Coimbra, Portugal

LUÍS PAQUETE, University of Coimbra, Portugal

The hypervolume indicator is one of the most used set-quality indicators for the assessment of stochastic multiobjective optimizers, as well as for selection in evolutionary multiobjective optimization algorithms. Its theoretical properties justify its wide acceptance, particularly the strict monotonicity with respect to set dominance, which is still unique of hypervolume-based indicators. This article discusses the computation of hypervolume-related problems, highlighting the relations between them, providing an overview of the paradigms and techniques used, a description of the main algorithms for each problem, and a rundown of the fastest algorithms regarding asymptotic complexity and runtime. By providing a complete overview of the computational problems associated to the hypervolume indicator, this article serves as the starting point for the development of new algorithms and supports users in the identification of the most appropriate implementations available for each problem.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Theory of computation** → **Algorithm design techniques**; **Computational geometry**; • **Applied computing** → **Multi-criterion optimization and decision-making**;

Additional Key Words and Phrases: Hypervolume indicator, hypervolume contributions, hypervolume subset selection problem, multiobjective optimization

ACM Reference format:

Andreia P. Guerreiro, Carlos M. Fonseca, and Luís Paquete. 2021. The Hypervolume Indicator: Computational Problems and Algorithms. *ACM Comput. Surv.* 54, 6, Article 119 (July 2021), 42 pages.

<https://doi.org/10.1145/3453474>

This work is financed by national funds through the FCT – Foundation for Science and Technology, I.P. within the scope of the project CISUC – UID/CEC/00326/2020, and was partially carried out in the scope of project MobiWise: From Mobile Sensing to Mobility Advising (P2020 SAICTPAC/0011/2015), co-financed by COMPETE 2020, Portugal 2020-POCI, European Regional Development Fund of the European Union and FCT. A. P. Guerreiro acknowledges FCT for Ph.D. studentship SFRH/BD/77725/2011, co-funded by the European Social Fund and by the State Budget of the Portuguese Ministry of Education and Science in the scope of NSRF–HPOP–Type 4.1–Advanced Training, and financial support by national funds through FCT with references PTDC/CCI-COM/31198/2017 and UIDB/50021/2020.

Authors' addresses: A. P. Guerreiro, CISUC, Department of Informatics Engineering, University of Coimbra, Polo II, 3030-290, Coimbra, Portugal, INESC-ID, Rua Alves Redol, 9, 1000-029, Lisbon, Portugal; email: andrea.guerreiro@tecnico.ulisboa.pt; C. M. Fonseca, CISUC, Department of Informatics Engineering, University of Coimbra, Polo II, 3030-290, Coimbra, Portugal; email: cmfonsec@dei.uc.pt; L. Paquete, CISUC, Department of Informatics Engineering, University of Coimbra, Polo II, 3030-290, Coimbra, Portugal; email: paquete@dei.uc.pt.



This work is licensed under a [Creative Commons Attribution International 4.0 License](https://creativecommons.org/licenses/by/4.0/).

© 2021 Copyright held by the owner/author(s).

0360-0300/2021/07-ART119

<https://doi.org/10.1145/3453474>

1 INTRODUCTION

In multiobjective optimization, a single optimal solution seldomly exists due to the (usually) conflicting nature of objectives. Instead, there is typically a set of Pareto-optimal solutions. The set of all such solutions (in decision space) is known as the Pareto-optimal set, and the corresponding images in objective space is known as the Pareto front. In such a case, the best solution of a given problem depends on the (subjective) preferences of the **Decision Maker (DM)**. As the Pareto front may be very large, and even infinite, the aim of optimizers under a no-preference information scenario is to find, and present to the DM, a set of solutions whose corresponding point set in objective space is a representative and finite subset of the Pareto front. In the search for such a subset, comparing the point sets to which different solution sets map becomes inevitable, and thus, also the need to define preferences over point sets. It is commonly accepted that the quality of a point set should be evaluated based on its closeness to the Pareto front (the closer the better), on the diversity in the set (the more evenly distributed they are, the better), and its spread [92].

Set-quality indicators facilitate the evaluation process of Pareto-front approximations by reconciling, in a single real value, characteristics such as proximity to the Pareto front, and diversity. Even though different set-quality indicators possibly value different characteristics, the knowledge of an indicators' properties is essential to the understanding of its inner preferences, and consequently, allow for a more conscious choice of the most appropriate quality indicator. Given its easy interpretation and its good properties, the hypervolume indicator rapidly became, and still is, one of the most widely used quality indicators among the many existing indicators [66].

The hypervolume set-quality indicator maps a point set in $\mathbb{R}^d$ to the measure of the region dominated by that set and (assuming minimization) bounded above by a given reference point, also in $\mathbb{R}^d$, where d is the number of objectives. It was first referred to as the "size of the space covered" [95, 96], and as "size of the dominated space" [90]. Alternative designations have also been used, such as $\mathcal{S}$ -metric [10, 96] and "Lebesgue measure" [42]. Different definitions of this indicator have been proposed. For example, it has been defined based on the union of polytopes [96] and, more generally based on the (integration of the) attainment function [47, 91]. The problem of computing the hypervolume indicator is known to be a special case of **Klee's Measure Problem (KMP)** [8], which is the problem of measuring the region resulting from the union of axis-parallel boxes. The hypervolume indicator is, in fact, a special case of KMP on unit cubes, and of KMP on grounded boxes [88]. See Reference [19] for a review on KMP's special cases and their relation to one another.

The hypervolume indicator was first proposed as a method for assessing multiobjective optimization algorithms [96]. It evaluates the optimizer outcome by simultaneously taking into account the proximity of the points to the Pareto front, diversity, and spread. The indicator's unique properties quickly led to its integration in **Evolutionary Multiobjective Optimization Algorithms (EMOAs)**, as a bounding method for archives [61], an environmental selection method [38], a ranking method [54], and as a fitness assignment method [5, 94, 97]. The integration of preferences in the indicator [2, 29, 91] has also been the subject of discussion, and so has the integration of diversity in the decision space [80]. Currently, the hypervolume indicator is one of the indicators used in the **Black-Box Optimization Benchmarking (BBOB)** tool [30] to continuously evaluate the external archive containing all nondominated solutions EMOAs generate during their execution. More recently, the hypervolume indicator has raised interest within the context of exact optimization algorithms [74, 75].

The merits of the hypervolume indicator are well recognized, however, its main drawback lies in its computational cost. This is particularly relevant as hypervolume-based EMOAs and benchmarking tools such as BBOB depend heavily on its computation. This imposes strong limitations

on the number of objectives considered and/or on EMOA parameters such as the number of generations and number of offspring. To overcome such a limitation, approximation algorithms [5] have been proposed, as well as objective reduction methods [31].

The main goal of this article is, first, to instigate the development of new algorithms for hypervolume-based problems by providing a broad overview of the current computational approaches to solving hypervolume-based problems, and by highlighting the intrinsic relation between the problems, which can be exploited. Second, this review is meant to promote best practices by providing summaries of the currently fastest algorithms for each problem, both regarding asymptotical and runtime performance, and by providing links to available implementations. This promotes the use of the most adequate algorithm either for application purposes, i.e., to make a more efficient use of hypervolume-based problems, and for benchmarking purposes, for example, to have fairer/adequate comparison tests with hypervolume-based EMOAs.

In the following sections, the theoretical advantages and the computational aspects of hypervolume indicator are discussed in more detail, mostly in the context of EMOAs. In Section 2, the hypervolume indicator and some related problems are formally defined, and its properties are reviewed. A review of the state-of-the-art algorithms for hypervolume-related problems is provided in Sections 3 to 6. The techniques and paradigms used in such algorithms are described first in Section 3. Then, Sections 4 and 5 provide a detailed overview of the existing algorithms for computing the hypervolume indicator and hypervolume contributions, respectively, and Section 6 briefly overviews the existing exact and approximation algorithms for the **Hypervolume Subset Selection Problem (HSSP)**. For a quick overview of the available and the fastest algorithms both with respect to runtime and to asymptotic complexity, see the “Remarks” sections: 4.2, 5.6, 6.1.1, and 6.2.1. Links to available implementations and to typical datasets are provided in Appendix A and B, respectively. Finally, concluding remarks are drawn in Section 7.

To make the referencing of algorithms easier throughout the text, the state-of-the-art algorithms are identified by a name and also alphanumerically. A letter and a number (e.g., [A2]) are assigned to each algorithm. The letter identifies to which problem the algorithm relates to (A, hypervolume indicator; B, hypervolume contributions; C, HSSP; D, (greedy) approximation to the HSSP) and the number is assigned according to the order in which they are introduced. For example, algorithm [B4] is the fourth algorithm related to hypervolume contributions (Section 5).

2 HYPERVOLUME-RELATED PROBLEMS

2.1 Notation

Spaces. (x, y) -, (x, y, z) -, and (x, y, z, w) -spaces will be referred to as, two-, three-, and four-dimensional spaces or, for brevity, as 2D, 3D, and 4D spaces, respectively.

Problem size. The lowercase letter n is used for the problem size, which is typically the size of the input set.

Number of dimensions. The lowercase letter d is used to represent the number of dimensions considered.

Points and sets. Points are represented by lowercase letters (in italics) and sets by Roman capital letters. For example, $p, q \in \mathbb{R}^d$ and $X, S \subset \mathbb{R}^d$.

Coordinates (for $d \leq 4$). Letters x, y, z , and w in subscript denote the coordinates of a point in an (x, y, z, w) -space. This notation is used only for spaces up to four dimensions. For example, if $p \in \mathbb{R}^3$, then $p = (p_x, p_y, p_z)$.

Coordinates (general case $d \geq 2$). For general, d -dimensional spaces, an index in subscript is used to identify the coordinate. For example, if $p \in \mathbb{R}^d$, then $p = (p_1, p_2, \dots, p_d)$, where p_i denotes the i -th coordinate of p , $i \in \{1, \dots, d\}$.

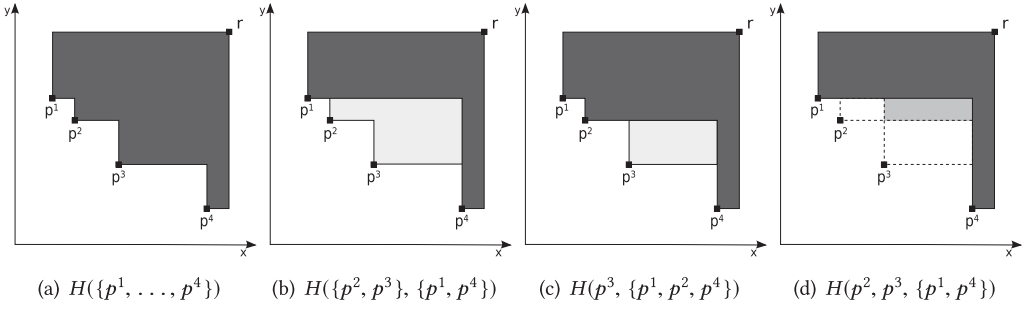


Fig. 1. Examples in two dimensions: (a) Hypervolume indicator (dark gray region), (b) hypervolume contribution of a point set (light gray region), (c) hypervolume contribution of a point (light gray region), and (d) joint hypervolume contribution (mid gray region).

Enumeration. Numbers in superscript are used to enumerate points or sets, e.g., $p^1, p^2, p^3 \in \mathbb{R}^d$ and $S^1, S^2 \subset \mathbb{R}^d$.

Projections. Projection onto $(d-1)$ -space by omission of the last coordinate is denoted by an asterisk. For example, given the point set $X = \{p, q\} \subset \mathbb{R}^3$, p^* and X^* denote the projection of the point p and of the point set X on the (x, y) -plane, respectively, i.e., $p^* = (p_x, p_y)$ and $X^* = \{(p_x, p_y), (q_x, q_y)\}$.

Dominance. A point $p \in \mathbb{R}^d$ is said to *weakly dominate* a point $q \in \mathbb{R}^d$ if $p_i \leq q_i$ for all $1 \leq i \leq d$. This is represented as $p \leq q$. If, in addition $q \not\leq p$, then p is said to (*strictly*) *dominate* q , which is represented here as $p < q$. If $p_i < q_i$ for all $1 \leq i \leq d$, then p is said to *strongly dominate* q , and this is represented as $p \ll q$.

2.2 Definitions

The hypervolume indicator [61, 95] is formally defined as follows:

Definition 2.1 (Hypervolume Indicator). Given a point set $S \subset \mathbb{R}^d$ and a reference point $r \in \mathbb{R}^d$, the hypervolume indicator of S is the measure of the region weakly dominated by S and bounded above by r , i.e.,

$$H(S) = \Lambda(\{q \in \mathbb{R}^d \mid \exists p \in S : p \leq q \text{ and } q \leq r\}),$$

where $\Lambda(\cdot)$ denotes the Lebesgue measure. Alternatively, it is interpreted as the measure of the union of boxes:

$$H(S) = \Lambda\left(\bigcup_{\substack{p \in S \\ p \leq r}} [p, r]\right),$$

where $[p, r] = \{q \in \mathbb{R}^d \mid p \leq q \text{ and } q \leq r\}$ denotes the box delimited below by $p \in S$ and above by r .

Since a fixed reference point, r , is assumed throughout this article, it is omitted as an argument of $H(\cdot)$ function. Figure 1(a) shows a two-dimensional example of the hypervolume (an area) and Figure 2(a) shows a three-dimensional example (a volume).

The hypervolume contribution of a point set to some reference point set [25, 27] is formally defined based on the definition of hypervolume indicator:

Definition 2.2 (Hypervolume Contribution of a Point Set). Given two point sets $X, S \subset \mathbb{R}^d$, and a reference point $r \in \mathbb{R}^d$, the (hypervolume) contribution of X to S is as follows:

$$H(X, S) = H(X \cup S) - H(S \setminus X).$$

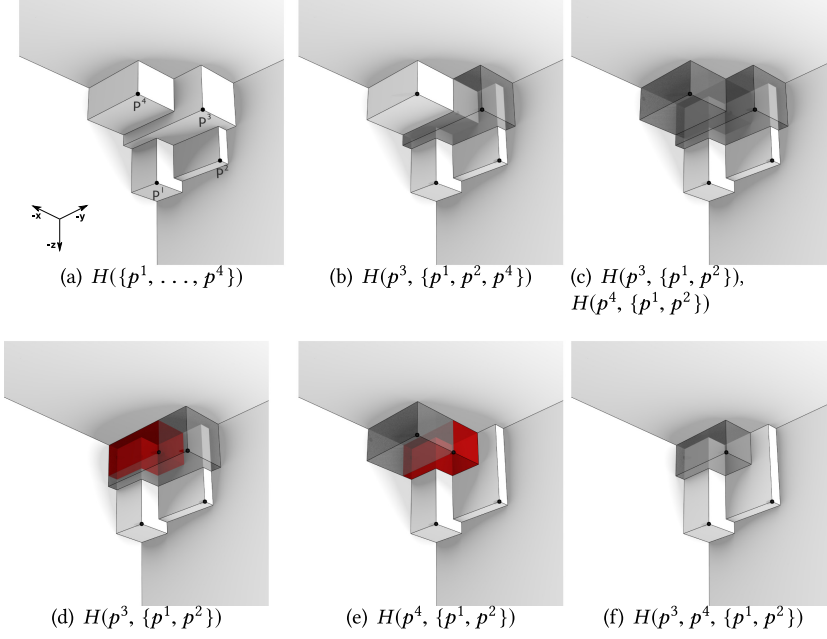


Fig. 2. Three-dimensional examples: (a) Hypervolume indicator (opaque volume), (b)–(c) hypervolume contribution (transparent volume), (d)–(f) joint contribution of p^3 and p^4 to $S = \{p^1, p^2\}$. Transparent red highlights the part of a contribution also dominated by the omitted point.

Note that if $X \cap S = \emptyset$, then the contribution of X to S is simply $H(X, S) = H(X \cup S) - H(S)$. See Figure 1(b) for a two-dimensional example. The particular case of $|X| = 1$ is used more frequently and is defined as in Reference [25]:

Definition 2.3 (Hypervolume Contribution). The hypervolume contribution of a point $p \in \mathbb{R}^d$ to a set $S \subset \mathbb{R}^d$ is as follows:

$$H(p, S) = H(S \cup \{p\}) - H(S \setminus \{p\}).$$

The hypervolume *contribution* of a point is sometimes referred to in the literature as the *incremental* hypervolume or the *exclusive* hypervolume [84]. Moreover, the contribution of a point p to the empty set is sometimes called the *inclusive* hypervolume [84]. See Figures 1(c) and 2(b) for two- and three-dimensional examples of a hypervolume contribution, respectively.

As pointed out in Reference [25], the above Definition 2.3 is consistent with the case where $p \in S$, and the contribution is the hypervolume lost when p is removed from S , as well as with the case where $p \notin S$, and the contribution of p is the hypervolume gained when adding p to S . While this is certainly convenient, it does not reflect the fact that the hypervolume gained by “adding” a point p to a set already including it is zero. However, this last situation can be handled easily as a special case by checking whether S includes p before applying the definition.

In some cases, such as when determining the decrease in the contribution of a given point $p \in \mathbb{R}^d$ to a set $S \subset \mathbb{R}^d$ due to the addition of another point $q \in \mathbb{R}^d$ to S , it is also useful to consider the contribution dominated simultaneously and exclusively by two points [52].

Definition 2.4 (Joint Hypervolume Contribution). The joint hypervolume contribution of $p, q \in \mathbb{R}^d$ to $S \subset \mathbb{R}^d$ is as follows:

$$H(p, q, S) = H((S \setminus \{p, q\}) \cup \{p \vee q\}) - H(S \setminus \{p, q\}),$$

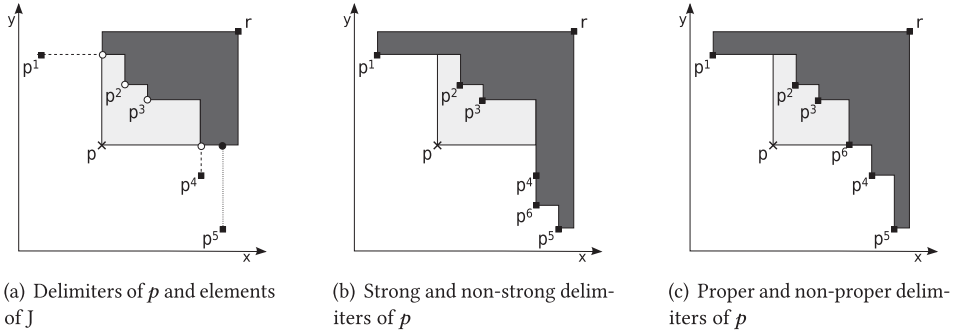


Fig. 3. Examples of the delimiters of the contribution of p to a point set. (a) Delimiters of p ($p^1, \dots, p^4$), as well as dominated point ($p \vee p^5$), represented by a filled circle, and nondominated points ($p \vee p^1$), ($p \vee p^2$), ($p \vee p^3$), ($p \vee p^4$), elements of J , represented by hollow circles (see text for more details), (b) strong ($p^1, \dots, p^3$) and non-strong (p^4, p^6) delimiters, (c) proper (p^1, p^2, p^3, p^6) and non-proper (p^4) delimiters.

where $\vee$ denotes the *join*, or component-wise maximum between two points. A general definition of the joint contribution to S of t points can also be found in the literature [46].

Figures 1(d) shows an example for the two-dimensional case and Figure 2(c)–(f) for the three-dimensional case. In the $d = 3$ example, Figure 2(c) shows the individual contribution of p^3 and p^4 to $S = \{p^1, p^2\}$, which partially overlap. This partially overlapped volume is the joint contribution (represented in transparent gray in Figure 2(f)). This joint contribution can also be interpreted as the region of the contribution of p^3 to S that is also dominated by p^4 (the transparent red region in Figure 2(d)). Analogously, it can be interpreted as the region of the contribution of p^4 to S that is also dominated by p^3 (the transparent red region in Figure 2(e)).

Moreover, the contribution of a point p to a set S is bounded above by certain points $q \in S$ that shall be referred to as *delimiters*, and are defined as follows [50]:

Definition 2.5 (Delimiter). Given a point set $S \subset \mathbb{R}^d$ and a point $p \in \mathbb{R}^d$, let $J = \text{nondominated}(\{(p \vee q) \mid q \in S \setminus \{p\}\})$. Then, $q \in S$ is called a (weak) *delimiter* of the contribution of p to S iff $(p \vee q) \in J$. If, in addition, $H(p, q, S) > 0$, then q is also a *strong delimiter* of the contribution of p to S .

Where $\text{nondominated}(X) = \{s \in X \mid \forall_{t \in X}, t \leq s \Rightarrow s \leq t\}$ denotes the set of nondominated points in X . Note that J is the smallest set of points weakly dominated by p that delimits its contribution to S , that is, $H(p, S) = H(p, J)$. Consequently, all $q \in J$ are such that $H(p, q, J) > 0$, and J is such that $H(p, S) = H(\{p\}) - H(J)$. Figure 3(a) shows an example where the contribution of p is delimited only by p^1, p^2, p^3 , and p^4 , where all of them are strong delimiters.

Non-strong delimiters can only exist when S contains points with repeated coordinates. In the example of Figure 3(b), p^1, p^2, p^3 are strong delimiters while p^4 and p^6 are not strong but only weak delimiters. This means that, in practice, only one point in such a group of delimiters is needed to bound the contribution of p . If one of them is deleted, then the contribution of p remains unchanged, whereas it increases if all are deleted.

The following extension to the notion of delimiter will also be needed in this work [50]:

Definition 2.6 (Outer Delimiter). Given a point set $S \subset \mathbb{R}^d$ and a point $p \in \mathbb{R}^d$, $q \in S$ is called an *outer delimiter* of the contribution of p to S if it is a delimiter of the contribution of p to $\{s \in S \mid p \not\leq s\}$. A delimiter, q , of the contribution of p to S is called an *inner delimiter* if it is not an outer delimiter, i.e., if $p \leq q$.

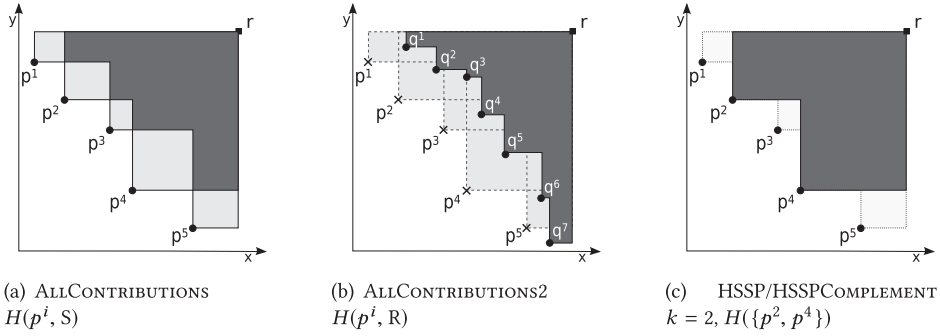


Fig. 4. Examples of hypervolume-related problems, given $S = \{p^1, \dots, p^5\}$, $R = \{q^1, \dots, q^7\}$, and $i = 1, \dots, 5$.

In general, outer delimiters may not be actual, or *proper*, delimiters in the sense of Definition 2.5, in particular, when p shares a coordinate with a point in S . In the example of Figure 3(c), points p^1, p^2, p^3, p^6 are the (proper) delimiters of the contribution of p to S , of which p^2, p^3 and p^6 are inner delimiters. There are two outer delimiters, p^1 and p^4 . Point p^4 is not a proper delimiter of the contribution of p to S , because $(p \vee p^6) = p^6 < (p \vee p^4)$.

2.3 Problems

Many computational problems related to the hypervolume indicator can be found in the literature. The following problems extend the list of problems given in References [39, 50]. Recall that the reference point is considered to be a constant.

PROBLEM 1 (HYPERVOLUME). Given an n -point set $S \subset \mathbb{R}^d$ and a reference point $r \in \mathbb{R}^d$, compute the hypervolume indicator of S , i.e., $H(S)$.

PROBLEM 2 (ONECONTRIBUTION). Given an n -point set $S \subset \mathbb{R}^d$, a reference point $r \in \mathbb{R}^d$ and a point $p \in \mathbb{R}^d$, compute the hypervolume contribution of p to S , i.e., $H(p, S)$.

PROBLEM 3 (ALLCONTRIBUTIONS). Given an n -point set $S \subset \mathbb{R}^d$ and a reference point $r \in \mathbb{R}^d$, compute the hypervolume contributions $H(p, S)$ of all points $p \in S$ to S .

PROBLEM 4 (ALLCONTRIBUTIONS2). Given an n -point set $S \subset \mathbb{R}^d$, an m -point set $R \subset \mathbb{R}^d$ such that $S \cap R = \emptyset$ and a reference point $r \in \mathbb{R}^d$, compute the hypervolume contributions $H(p, R)$ of all points $p \in S$ to R .

Note that, by definition, the contributions of two points $p, q \in S$ to S never overlap in Problem 3 while in Problem 4, the contributions of $p, q \in S$ to a point set R may overlap. For example, in Figure 4(b), given $S = \{p^1, \dots, p^5\}$ and $R = \{q^1, \dots, q^7\}$, the contribution of the point $p^2 \in S$ to R (Problem 4) includes all the lighter-gray regions dominated by p^2 , including regions partially dominated by p^1, p^3 , and p^4 . However, in Figure 4(a), the contribution of p^2 to S (Problem 3) corresponds solely to the respective exclusively dominated (lighter-gray) region. Due to this distinction, the definition of the greatest/least contributor of S depends on whether the contributions are computed with respect to the set S itself or to another set, R , as in Problems 3 and 4, respectively:

PROBLEM 5 (GREATEST/LEASTCONTRIBUTOR). Given an n -point set $S \subset \mathbb{R}^d$ and a reference point $r \in \mathbb{R}^d$, find a point $p \in S$ with maximal/minimal hypervolume contribution to S .

PROBLEM 6 (GREATEST/LEASTCONTRIBUTOR2). Given an n -point set $S \subset \mathbb{R}^d$, an m -point set $R \subset \mathbb{R}^d$ such that $S \cap R = \emptyset$ and a reference point $r \in \mathbb{R}^d$, find a point $p \in S$ with maximal/minimal hypervolume contribution to R .

Sometimes, the above problems are computed for a sequence of sets that differ in a single point from the previous one, either by adding a point to (Incremental case) or by removing a point from (Decremental case) the previous set.

PROBLEM 7 (UPDATEHYPERVOLUME). *Given an n -point set $S \subset \mathbb{R}^d$, the reference point $r \in \mathbb{R}^d$, the value of $H(S)$, and a point $p \in \mathbb{R}^d$, compute:*

- **Incremental:** $H(S \cup \{p\}) = H(S) + H(p, S)$, where $p \notin S$.
- **Decremental:** $H(S \setminus \{p\}) = H(S) - H(p, S)$, where $p \in S$.

PROBLEM 8 (UPDATEALLCONTRIBUTIONS). *Given an n -point set $S \subset \mathbb{R}^d$, a reference point $r \in \mathbb{R}^d$, the value of $H(q, S)$ for every $q \in S$, and a point $p \in \mathbb{R}^d$:*

- **Incremental:** Compute $H(q, S \cup \{p\}) = H(q, S) - H(p, q, S)$ for all $q \in S$, and also $H(p, S)$, where $p \notin S$.
- **Decremental:** Compute $H(q, S \setminus \{p\}) = H(q, S) + H(p, q, S)$ for all $q \in S \setminus \{p\}$, where $p \in S$.

PROBLEM 9 (UPDATEALLCONTRIBUTIONS2). *Given an n -point set $S \subset \mathbb{R}^d$, an m -point set $R \subset \mathbb{R}^d$, a reference point $r \in \mathbb{R}^d$, the value of $H(q, R)$ for every $q \in S$, and a point $p \in \mathbb{R}^d$:*

- **Incremental:** Compute $H(q, R \cup \{p\}) = H(q, R) - H(p, q, R)$ for all $q \in S$, where $p \notin R \cup S$.
- **Decremental:** Compute $H(q, R \setminus \{p\}) = H(q, R) + H(p, q, R)$ for all $q \in S$, where $p \in R$ and $p \notin S$.

Finally, based on the definition in Reference [5], the Hypervolume Subset Selection Problem¹ is formally defined here as:

PROBLEM 10 (HSSP). *Given a n -point set $S \subset \mathbb{R}^d$ and an integer $k \in \{0, 1, \dots, n\}$, find a subset $A \subseteq S$ such that $|A| \leq k$ and*

$$H(A) = \max_{\substack{B \subseteq S \\ |B| \leq k}} H(B).$$

The complement problem of the HSSP is defined as:

PROBLEM 11 (HSSPCOMPLEMENT). *Given a n -point set $S \subset \mathbb{R}^d$ and an integer $k \in \{0, 1, \dots, n\}$, find a subset $C \subseteq S$ such that $|C| \geq (n - k)$ and*

$$H(C, S) = \min_{\substack{B \subseteq S \\ |B| \geq (n-k)}} H(B, S).$$

If $A \subseteq S$ is a solution to the HSSP given k and S , then $S \setminus A$ is a solution to the HSSPCOMPLEMENT, and vice-versa. For example, in Figure 4(c), given $S = \{p^1, \dots, p^5\}$ and $k = 2$, the optimal solution to the HSSP is $\{p^2, p^4\}$, and the optimal solution to the HSSPCOMPLEMENT is $\{p^1, p^3, p^5\}$.

Note that, in the above problems, S is usually a nondominated point set, even though this is not mandatory. Any dominated point $q \in S$ has a null contribution to S . However, if q is dominated by a single point $p \in S$, then the contribution of p to S will be lower than what it would be if $q \notin S$. Moreover, the incremental scenarios of Problems 7 to 9 explicitly require that $p \notin S$, because the adopted definition of hypervolume contribution does not handle adding a point to a set in which it is already included, as discussed before, nor does it consider the multiset that would result from such an operation. If such cases become relevant, then the hypervolume contribution of repeated points in a multiset should be considered to be zero.

¹Note that the HSSP has also been defined in Reference [2] as the subset selection problem with respect to the Weighted Hypervolume Indicator of which the hypervolume indicator is a special case.

2.3.1 Relation between problems. Most of the problems listed above are not expected to be efficiently solved for an arbitrary number of dimensions d . For example, the **HYPERVOLUME** [21] and **ONECONTRIBUTION** [22] problems are known to be $\#P$ -hard. Even deciding if a point is the least contributor is $\#P$ -hard [22]. Moreover, **HSSP** was recently shown to be **NP**-hard [20] for $d \geq 3$. Although these are not encouraging results for an arbitrary d , this does not mean that efficient algorithms to compute hypervolume-related problems exactly, or to approximate the **HSSP** cannot be developed for a fixed and small d . To develop such efficient algorithms, it is important to understand how the various hypervolume problems relate to each other or if they arise as subproblems. All problems above (Problems 1 to 11) are intrinsically related. For most of them, it is possible to solve each one by solving one or more instances of the others. Consequently, state-of-the-art algorithms frequently exploit these relations.

It is clear from Definition 2.3 that any algorithm that computes **HYPERVOLUME** (Problem 1) can also be used to compute **ONECONTRIBUTION** (Problem 2). In fact, by Definition 2.5, it can be computed considering only p and its delimiters. Moreover, the **UPDATEHYPERVOLUME** problem (Problem 7) can be solved by computing either **HYPERVOLUME** given $S \cup \{p\}$ or **ONECONTRIBUTION** given S and p . Conversely, the **HYPERVOLUME** problem can be computed by solving a sequence of **UPDATEHYPERVOLUME** problems as new points are added to a set. For example, consider $S = \{p^1, p^2, p^3\}$, the hypervolume $H(S)$ can be computed as the sum $H(p^1, \{\}) + H(p^2, \{p^1\}) + H(p^3, \{p^1, p^2\})$, more generally:

$$H(S) = \sum_{i=1}^n H(p^i, \{p^1, \dots, p^{i-1}\}) = H(p^1, \{\}) + H(p^2, \{p^1\}) + \dots + H(p^n, \{p^1, \dots, p^{n-1}\}),$$

where $S = \{p^1, \dots, p^n\}$. In fact, when such points are sorted according to dimension d , then the sequence of subproblems become $(d - 1)$ -dimensional problems, as exploited by the dimension-sweep approach (see Section 3.1.2). Dedicated algorithms to solve **UPDATEHYPERVOLUME** (and the other update problems) can take advantage of previous calculations and data structures to avoid redundant (re)computations and consequently, to save time.

It should also be clear that any algorithm that computes **ONECONTRIBUTION** can be used to compute **ALLCONTRIBUTIONS** (Problem 3), and vice versa. Algorithms to solve **ALLCONTRIBUTIONS** can also be used to solve the **GREATEST/LEASTCONTRIBUTOR** (Problem 5), by computing all contributions and then selecting a point with maximal/minimal contribution, although it is not strictly required to know all contributions to find the least contributor (see **IHSO** [B5] algorithm in Section 5). In the absence of dedicated algorithms, **UPDATEALLCONTRIBUTIONS** (Problem 8) can be solved by recomputing all contributions (**ALLCONTRIBUTIONS**).

Analogously to Problem 3, algorithms to compute **ONECONTRIBUTION** can also be used to compute **ALLCONTRIBUTIONS2** (Problem 4) problem. Algorithms to solve **ALLCONTRIBUTIONS2** can also be used to solve the **GREATEST/LEASTCONTRIBUTOR2** (Problem 6), by computing the contribution of each point in S to the set R and then selecting a point with maximal/minimal contribution. Moreover, **UPDATEALLCONTRIBUTIONS2** (Problem 9) can also be solved by recomputing the contributions (**ALLCONTRIBUTIONS2**). Despite the similarities, **ALLCONTRIBUTIONS** and **ALLCONTRIBUTIONS2** are distinct to the point that one cannot be directly used to solve the other. The same observation applies to the corresponding update problems (**UPDATEALLCONTRIBUTIONS** and **UPDATEALLCONTRIBUTIONS2**).

The definition of **HSSP** suggests that it can be solved by enumerating all subsets of size k and computing the hypervolume indicator of each one. Similarly, **HSSPCOMPLEMENT** can be computed in an analogous way. However, this is obviously not practical as the time required would quickly be unacceptable unless n is sufficiently small and k is either very small or close to n .

Moreover, recall that an optimal solution to the HSSP can be obtained from an optimal solution to HSSPCOMPLEMENT and vice versa. For the particular case of $k = n - 1$, the LEASTCONTRIBUTOR problem provides the solution to the HSSP by definition.

Finally, it is important to have in mind that the way problems are solved has effect on the precision of the calculations. As pointed out in Reference [73], computing the contribution of a point as the subtraction of two large hypervolumes raises precision problems. It is thus recommended to avoid subtracting hypervolumes as much as possible and, ideally, performing subtractions only between coordinates. For example, regarding numerical stability, using a specific algorithm to compute ONECONTRIBUTOR may be preferable to using an algorithm for HYPERVOLUME to compute the contribution based on a subtraction.

Section 3 gives more details on the relation between hypervolume-based problems by explaining the existing algorithms and the techniques used by them.

2.3.2 Applications in EMOAs. Different hypervolume-related problems are useful at different stages of EMOAs. The most common application of the HYPERVOLUME problem is for performance assessment [30, 95]. In such a case, the hypervolume is computed for the image of the solution sets returned by EMOAs considering a given (fixed) reference point, and the greater the hypervolume value is, the better a solution set is considered to be. Monitoring the quality of an (unbounded) archive or of a population through generations can be done by solving the HYPERVOLUME problem [30] or, in steady-state scenarios (only one solution is created and only one is discarded at each new generation), by solving the UPDATEHYPERVOLUME problem. As hypervolume contributions can be viewed as the increment in quality that (the image of) a solution adds to a set, the ALLCONTRIBUTIONS problem finds application in fitness assignment [5, 97], and the update counterpart (UPDATEALLCONTRIBUTIONS problem) is useful to update such contributions in steady-state scenarios. The sequential computation of LEASTCONTRIBUTOR or of GREATESTCONTRIBUTOR2 can also be used for ranking solutions [54]. The view of environmental selection as a subset selection problem where the goal is to select the best subset of solutions according to some measure, makes the HSSP (and the HSSPCOMPLEMENT) well fit for this purpose [38, 61, 82]. In a steady-state scenario this consists of solving a LEASTCONTRIBUTOR problem [38], which can be done through an UPDATEALLCONTRIBUTIONS [50] problem.

The idea of using the update version of the problems, particularly in steady-state scenarios, is to take advantage of the iterative nature of EMOAs and the small modifications done to the archive and/or population at each iteration, i.e., to use information from previous generations (e.g., data structures and known contributions) to save time. Note however that the EMOAs do not need to be aware of such data structures, as the update algorithms are responsible for keeping track of the state of the computations and to provide procedures (see, for example, the algorithms in Section 5.4) that allow the EMOA to order such updates.

2.4 Properties

As an indicator that imposes a total order among point sets, the hypervolume indicator is biased towards some type of distribution of the points on a front [91]. Understanding that bias by studying its properties allows to better understand the underlying assumptions on DM preferences. Among the most important properties are the monotonicity properties, sensitivity to objective rescaling and parameter setting, and optimal μ -distributions (see Reference [93] for an overview of properties of quality indicators). In particular, monotonic properties reflect the formal agreement between a binary relation on point sets and the ranking imposed by a unary set-quality indicator. Given a binary relation R on point sets, a set-quality indicator I (to be maximized) is said to be weakly R -monotonic if $A R B$ implies $I(A) \geq I(B)$ for any $A, B \subset \mathbb{R}^d$, and it is strictly

R -monotonic if ARB implies $I(A) > I(B)$. The study of optimal μ -distributions describe how points in an indicator-optimal subset of maximum size μ are distributed in the objective space given a known Pareto Front.

The hypervolume indicator is well acknowledged by its properties. It is scaling independent [60, 90], i.e., the ranking over sets does change when a strictly monotonic transformation is applied to one of the objectives. It is strictly $<$ -monotonic [60, 90], where $A < B$ (A strictly dominates B) if and only if for every point $b \in B$ there is a point $a \in A$ that weakly dominates it, but not the other way around.

This implies that this indicator is maximal for the Pareto front [42, 49] and that scaling objectives does not affect the order imposed on point sets. Moreover, hypervolume-based selection methods provide desirable (convergence) properties to EMOAs, see References [25, 68] for more details.

Concerning optimal μ -distributions in two dimensions, the exact location of the points in an optimal subset of a given size μ is known only for continuous linear fronts [3]. In this case, there is a unique optimal subset where all points are on the Pareto front and uniformly spaced between two outer points, the position of which depends both on the reference point and on the two extreme points of the Pareto front [28]. General fronts have also been studied, but only in terms of point density on the Pareto front when the number of points μ tends to infinity [3]. Unfortunately, the results available for the two-objective case do not generalize easily to three objectives, and not much is known about optimal μ -distributions in this case. The main results concern whether there exists a setting of the reference point that guarantees the inclusion of a front's extreme points in the optimal μ -distribution [1, 78] and the derivation of the optimal μ -distributions for the special case of Pareto fronts consisting of a line segment embedded in a three-objective space [78]. Results on more general fronts have only been obtained empirically [79].

The theoretical studies already emphasize that the location of the reference point influences the optimal μ -distributions and the inclusion/exclusion of boundary points [3, 28]. Empirical studies further indicate that, in cases with more than two objectives, setting the reference point too far from the front may lead to subset of points in the boundary of the front, whereas a closer reference point may lead to subsets of points spread more evenly all over the front [55]. Therefore, setting the reference point in such a way that the obtained distributions are in agreement with the DM preferences is not straightforward. See Reference [55] for recommended reference point specifications when seeking evenly distributed point sets, and Reference [77] for a more detailed overview on aspects concerning the usage of the hypervolume indicator in evolutionary multiobjective optimization.

Ulrich and Thiele [81] showed that the hypervolume indicator is a submodular function. Given a decision space X and a function $z : 2^X \rightarrow \mathbb{R}$, z is submodular if

$$\forall A, B \subseteq X, \quad z(A) + z(B) \geq z(A \cup B) + z(A \cap B). \quad (1)$$

Submodularity is an important property as it relates to convexity in combinatorial optimization [69]. Additionally, a submodular function z is non-decreasing (or monotone) if

$$\forall A \subseteq B \subseteq X, \quad z(A) \leq z(B). \quad (2)$$

The hypervolume indicator is a non-decreasing submodular function [81]. See Reference [70] for alternative equivalent definitions of (non-decreasing) submodular functions and examples. Because the HSSP consists of maximizing a submodular function subject to a cardinality constraint [44], the approximation of HSSP by means of a (incremental) greedy approach has an approximation guarantee [70]. Hence, the subset obtained by selecting k points from S one at a time so as to maximize the hypervolume gained at each step is a $(1 - 1/e)$ -approximation to the hypervolume of an optimal subset, i.e., the ratio between the greedy solution and the optimal solution is greater than

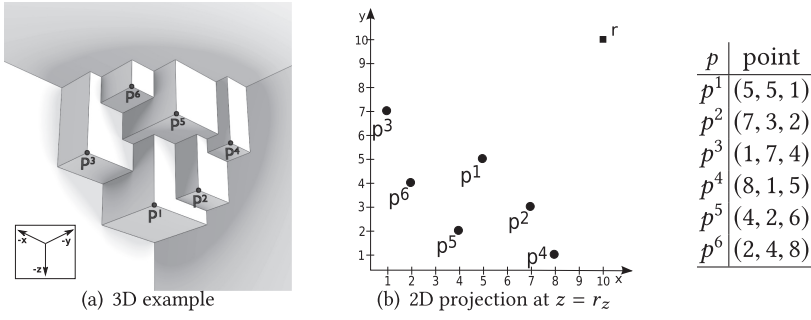


Fig. 5. Three-dimensional base example and the corresponding projection on the (x, y) -plane. The reference point is $r = (10, 10, 10)$.

or equal to $(1 - 1/e) \approx 0.63$. A tighter approximation bound for $k > \frac{n}{2}$ is known [65], which takes into account that in such case a greedy and an optimal solution must agree in m points where m is at least $2k - n$. The new bound relies both in n and k while the previous $(1 - 1/e)$ bound did not. A weak but simple form of the new bound is: $1 - (1 - \frac{m}{k})(1 - \frac{1}{k})^{k-m}$. Moreover, an approximation guarantee of $\frac{k}{n}$ for the approximation of HSSP by means of a decremental greedy approach was independently derived specifically for the HSSP [49] and more generally for the maximization of monotone submodular functions subject to a cardinality constraint [76]. In this case the subset is obtained by discarding $n - k$ points from X one at a time so as to minimize the hypervolume lost at each step. Such approximation ratios do not extend to the HSSPCOMPLEMENT counterpart, since the approximation ratio with either approaches can be arbitrarily large [24, 50].

3 PARADIGMS AND TECHNIQUES

3.1 Paradigms

Figure 5 shows a three-dimensional example that is used for illustration purposes throughout this section. Moreover, n is used to represent the input size.

3.1.1 Inclusion-Exclusion Principle. The inclusion-exclusion principle is a technique consisting of sequentially iterating over an inclusion step followed by an exclusion step, where the i -th step involves some computation for every combination of i points. This technique is discussed in Reference [4] for the HYPERVOLUME problem: The hypervolume indicator of a set $S \subset \mathbb{R}^d$ of n points is the sum of the hypervolume (indicator) of every subset with a single point, minus the hypervolume of the component-wise maximum of each pair of points, plus the hypervolume of the component-wise maximum of each subset with three points, minus the hypervolume of the component-wise maximum of each subset of four points, and so on. This technique can be very inefficient, if applied as explained above it is exponential in the number of points, $\Theta(2^n)$.

3.1.2 Dimension Sweep. Dimension sweep [71] is a paradigm that has been widely used in the development of algorithms for hypervolume-related problems (e.g., [A2], [A5], [A3], [A9], and [A10]). A problem involving n points in $\mathbb{R}^d$ is solved with this paradigm by visiting all points in ascending (or descending) order of one of the coordinates, solving a $(d - 1)$ -dimensional subproblem for each point visited, and combining the solutions of those subproblems. The subproblems themselves can often be solved using dimension sweep as well, until a sufficiently low-dimensional base case is reached, which can be solved easily by a dedicated algorithm. However, the time complexity of the resulting algorithms typically increases by an $O(n)$ factor per dimension.

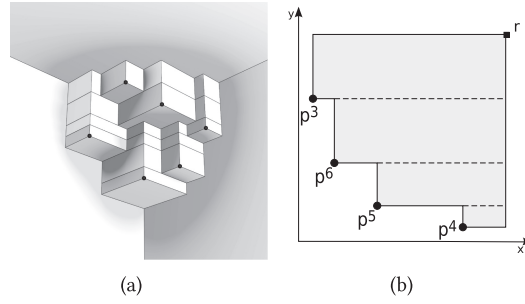


Fig. 6. Example of: (a) the slice division of the volume in Figure 5(a), and (b) the area of the corresponding topmost slice.

A typical dimension-sweep algorithm for **HYPERVOLUME** problem works as follows. Input points are sorted and visited in ascending order of the last coordinate. The d -dimensional dominated region is partitioned into n slices by axis-parallel cut hyperplanes defined by the last coordinate value of each input point and the reference point. The desired hypervolume indicator value is the sum of the hypervolumes of all slices, and the hypervolume of a slice is the hypervolume of its $(d - 1)$ -dimensional base multiplied by its height. The base of a slice is the $(d - 1)$ -dimensional region dominated by the projection of the points below it according to dimension d onto the corresponding cut hyperplane. The height of a slice is the difference between the values of the last coordinate of two consecutive points.

Figure 6(a) exemplifies how the volume in the example of Figure 5 could be split into $n = 6$ (horizontal) slices. This splitting and the computation of the volume for this example is further detailed in the explanation of HV3D (see [A9]) in Section 4. Figure 6(b) shows a splitting of the base area of the topmost slice in Figure 6(a).

The algorithms for hypervolume-based problems using dimension sweep differ mostly in the $(d - 1)$ -dimensional subproblem considered. For example, for the **HYPERVOLUME** problem, one option would be to compute the $(d - 1)$ -dimensional hypervolume indicator of the base of the slice from scratch, while another would be to avoid the full computation by updating the hypervolume of the base of the previous slice.

3.1.3 Spatial Divide-and-Conquer. This technique consists of splitting the d -dimensional hypervolume into two (or more) parts and recursively solving each part until a problem easy to solve is reached. For example, the problem may be split into two subproblems according to the median point, p , of a given dimension i , i.e., the $\lceil (n + 1)/2 \rceil$ -th point with lowest coordinate i . The axis parallel hyperplane at the value p_i of coordinate i divides the hypervolume in two parts. The first part refers to a subproblem containing the $\lfloor n/2 \rfloor$ points below p in the i -th coordinate and the reference point is the one of the current problem but projected on the splitting hyperplane. The second subproblem contains all n points, but the $\lfloor n/2 \rfloor$ points below p in the i -th coordinate are projected onto the splitting hyperplane. Figure 7 shows an example of these two subproblems where coordinate $i = 3$ is used for splitting and the median point p^4 is the splitting point p .

This approach is used, for example, by HOY algorithm (see [A4] in Section 4) to compute **HYPERVOLUME** and by Bringmann and Friedrich's algorithm (see [B1] in Section 5) to compute **ALLCONTRIBUTIONS**. An orthogonal partition tree is typically used as the underlying data structure, as in the mentioned algorithms. In that case, the hyperrectangle bounded below by the component-wise minimum of the initial point set and above by the reference point is recursively partitioned in axis parallel regions, and each one of them is associated to a node. Moreover, the typical base

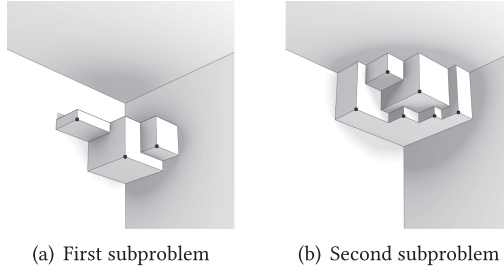


Fig. 7. Example of the volume division in Figure 5(a) using the spatial divide-and-conquer approach.

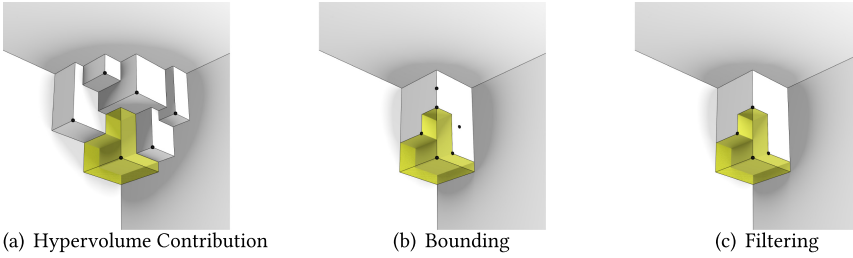


Fig. 8. Example of the bounding technique for the contribution of p^1 in Figure 5(a).

case of the recursion (a leaf of the partition tree) occurs when a partition consisting of a trellis is reached. A trellis is a region (hyperrectangle) where every point in the subproblem dominates that region in all coordinates except one [8].

The Multidimensional Divide-and-Conquer [7] is a different type of divide-and-conquer that divides the problem into two d -dimensional subproblems of size $n/2$ and one $(d - 1)$ -dimensional subproblem of size n (the merge step). This paradigm is mentioned only for completeness, as the $O(n \log n)$ HVDC3D [48] algorithm for the $d = 3$ case of the HYPERVOLUME problem appears to be the only algorithm for hypervolume-based problems based on this paradigm.

3.2 Techniques

3.2.1 Bounding Technique. This technique consists of projecting points onto the surface of an axis parallel d -dimensional box [16, 23] (bounding step) and discarding the points that become dominated (filtering step), in d -dimensional space. In practice, it consists of determining the auxiliary set J in the definition of delimiter (see Definition 2.5), where J is the smallest set of points weakly dominated by p that delimits its contribution. It is used in several algorithms (e.g., References [67, 84, 85]), in particular, to compute hypervolume contributions. Note that, to explicitly compute J , the bounding step requires $O(n)$ time, and the filtering step can be performed in $O(n \log^{\max(1, d-2)} n)$ time [7].

See Figure 8 for an example. Figure 8(a) shows the hypervolume of the point set $S = \{p^2, \dots, p^6\}$ from Figure 5 and the hypervolume contribution of p^1 (in transparent yellow). Figure 8(b) shows the bounding step, where all points (not dominated by p^1) are projected on the surface of the region dominated by p^1 , and where the gray axis-parallel box shown is the volume dominated by such projections. Only the nondominated points among those projected points are kept (see Figure 8(c)).

This technique is used in the computation of hypervolume contributions, because these projections are enough to delimit the contribution of p^1 and because the absolute position of the delimiters resulting in those projections is irrelevant, i.e., have no influence on the contribution of

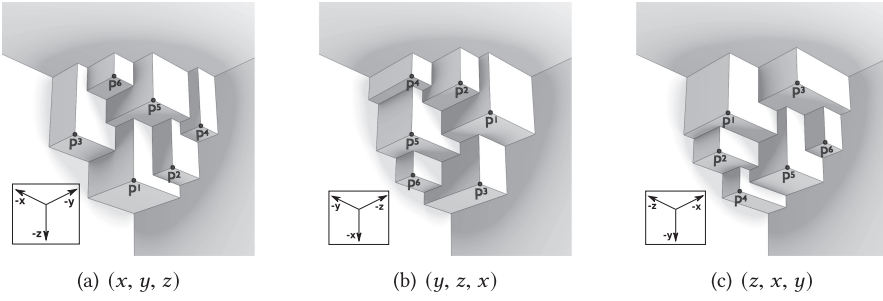


Fig. 9. Example of Figure 5(a) considering three of the six objective order possible.

p^1 . Moreover, this bounding technique allows to further discard the points that are not delimiters of the contribution of p^1 as they are unnecessary to compute it.

3.2.2 Objective Reordering. Problems in computational geometry such as those related to the hypervolume indicator are invariant with respect to objective reordering and, in particular, to the case where it is equivalent to rotation. See the example in Figure 9 of a volume in (x, y, z) coordinate system (see Figure 9(a)) and its counterclockwise (see Figure 9(b)) and clockwise rotations (see Figure 9(c)), corresponding to the (y, z, x) and (z, x, y) coordinate systems, respectively. Reordering objectives is important, since, although the result of the computation does not change, it can have an impact on the implementation runtime [86]. For example, dimension-sweep approaches are usually sensitive to objective ordering as a particular order may result in more dominated points further in the recursion than others. While et al. [86] proposed several heuristics to determine the best objective order to consider. Algorithms such as **Walking Fish Group (WFG)** algorithm (see Section 4) benefit with the integration of such heuristics, by repeating it in several steps of the computation of the hypervolume indicator, which results in fast algorithms in practice in many dataset instances.

3.2.3 Local Upper Bounds. In the context of multiobjective optimization, the *search region* [59] is understood as a promising region in the objective space where more optimal solutions can be found. It is therefore disjoint with respect to the region dominated by a set of already found non-dominated points, $S \subset \mathbb{R}^d$, and bounded from above by a reference point, $r \in \mathbb{R}^d$. Recall that the hypervolume indicator can be defined as the union of boxes bounded below by a point in the non-dominated point set and above by the reference point. Analogously, the search space is also defined by the union of boxes that are unbounded below and bounded above by *local upper bounds* [58]. The set of local upper bounds can be roughly defined as the nondominated point set of the search region considering maximization (see Figure 10, where the white region is contained in the search region). These local upper bounds can be computed from the given set S of n nondominated points and the reference point r . In $d = 2$ there are $n + 1$ local upper bounds and in $d = 3$ there are $2n + 1$ [36]. In the general $d \geq 2$ case there are $\Theta(n^{\lfloor d/2 \rfloor})$ upper bounds [58, 59]. Such local upper bounds can be determined and used for the computation of the hypervolume indicator of S (see [A8] in Section 4).

3.2.4 Dealing with Dominated Points. Given a point set $S \subset \mathbb{R}^d$, if there is a dominated point $q \in S$, then it has zero hypervolume contribution to S . However, if it is dominated by a single point, $p \in S$, then q is an inner delimiter of the contribution of p to S , which implies that if q is removed from S then the contribution of p increases. Therefore, point q is needed when computing the contribution of p to S . There are mainly two ways to deal with dominated points in this scenario:

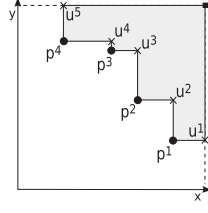


Fig. 10. Example of local upper bounds (points $u^1, \dots, u^5$ represented by crosses, $\times$) given the nondominated point set $S = \{p^1, \dots, p^4\}$.

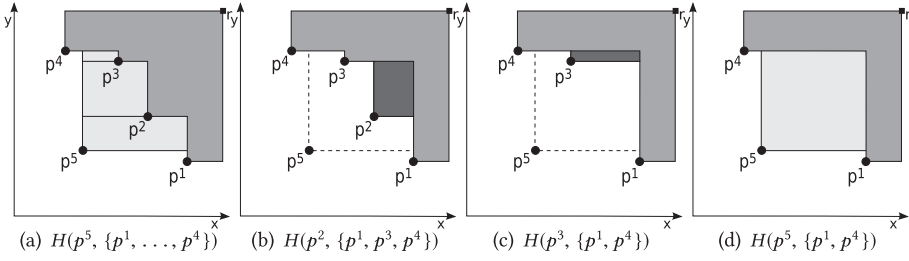


Fig. 11. Computation of the contribution of p^5 to $\{p^1, \dots, p^4\}$ either (a) by splitting it in slices or (b)–(d) by previously removing the contribution of the points dominated by p^5 .

one that preserves the structure of the contribution of p to S and one that destroys it. The latter is typically a work-around that allows to use algorithms that assume a nondominated point set as input.

Figure 11 shows an example where $S = \{p^1, \dots, p^5\}$. Assume that the point set $\{p^1, \dots, p^4\} \subset \mathbb{R}^2$ is kept sorted in a linked list. The contribution of $p = p^5$ to S can be computed in a structure-preserving manner by splitting the contribution of p in three horizontal (or vertical) slices, as in Figure 11(a), and summing up their areas. A structure-destructive manner would consist of computing the contribution of p^2 to $\{p^1, p^3, p^4\}$, removing p^2 from the list, computing the contribution of p^3 to $\{p^1, p^4\}$, removing p^3 from the list (see Figure 11(b) and (c)), then computing the contribution of p to $\{p^1, p^4\}$ (see Figure 11(d)). Subtracting the first two contributions from the latter one yields the contribution of p to S :

$$H(p^5, S) = H(p^5, \{p^1, p^4\}) - H(p^3, \{p^1, p^4\}) - H(p^2, \{p^1, p^3, p^4\}).$$

Note that the described structure-destructive method requires solving a sequence of $t + 1$ ONECONTRIBUTION problems, where t is the number of points dominated by p . In addition, the information concerning the inner delimiters, and consequently the shape, of the contribution of p to S is lost (see Figure 11(d)). Thus, structure-preserving methods may be more efficient, and more adequate, than structure-destructive ones particularly when it comes to update contribution(s). While the latter would likely have to recompute them from scratch, the former may save time by taking advantage of its data structures.

4 HYPERVOLUME INDICATOR

There are many algorithms to compute the HYPERVOLUME problem. This section overviews only the most conceptually distinct ones (that are based on a different paradigm/technique) and the fastest ones according either to their asymptotic complexity or to their runtime efficiency. The

list of algorithms excluded from this overview include some algorithms based on problems for which the HYPERVOLUME is a particular case (e.g., References [18, 32, 88]) and others specific to the HYPERVOLUME computation (e.g., References [4, 17]), approximation algorithms (e.g., References [21, 41, 57]), algorithms based on parallelization (e.g., References [67, 73]) and algorithms for cases with uncertain objective [40].

4.1 Algorithms

[A1] Lebesgue Measure Algorithm (LebMeasure). The LebMeasure algorithm [42] was one of the first algorithms proposed to compute HYPERVOLUME for any number of dimensions ($d > 1$). The algorithm maintains a list of points, $S \subset \mathbb{R}^d$, initially set to the input point set. The first step is to remove a point p from S and to compute and accumulate the hypervolume of an hypercube inside the region exclusively dominated by p . This is the hypercube bounded below by p and bounded above in every coordinate $i \in \{1, \dots, d\}$ by b_i , the lowest i -th coordinate value in the point set $\{q \in S \mid p_i \leq q_i\}$. To account for the part of the contribution of p to S not yet computed, d new points are added to S , each one is a projection of p onto an axis-parallel hyperplane at b_i in coordinate i . Then, dominated points are removed from S . These steps are repeated until S is empty. Although LebMeasure was presented as a polynomial-time algorithm, its time complexity was later shown to be exponential in d : $O(n^d)$ [83].

[A2] Hypervolume by Slicing Objectives (HSO). The HSO algorithm [60, 87]² is a direct application of the dimension sweep approach to the computation of HYPERVOLUME problem. HSO works exactly as explained in Section 3.1.2 and has $O(n^{d-1})$ -time complexity. Several algorithms were proposed based on HSO, considering different $(d - 1)$ -dimensional subproblems, data structures and/or base cases or by combining it with other techniques (e.g., FPL [A3] and WFG [A5]).

[A3] Fonseca, Paquete, and López-Ibañez's algorithm (FPL). The FPL algorithm [43] is based on HSO and has $O(n^{d-2} \log n)$ time complexity and $O(n)$ space complexity. FPL improves upon HSO through the use of more efficient data structures, the caching of previous computations and the use of a better base case for the recursion. In particular, FPL maintains points sorted according to every dimension in circular doubly linked lists. In an FPL recursion call, points are removed from (some) lists in decreasing order of a given dimension and are reinserted in reverse order. This behavior enables constant time insertions. Moreover, when a new point is visited in an i -th dimensional subproblem, it adds contribution only to the $(i - 1)$ -th dimensional slices above it in dimension $i - 1$. Thus, if $(i - 1)$ -th dimensional slices below that point have been previously computed then FPL has that information stored and does not need to recompute them as HSO does. Finally, FPL stops the recursion at $d = 3$ where it uses HV3D (see [A9]). Note that FPL may be improved with more efficient base cases. For example, if HV4D⁺ (see [A10]) is used as a base case for $d = 4$, the time complexity of FPL would improve to $O(n^{d-2})$ for $d \geq 4$.

[A4] Hypervolume Overmars and Yap (HOY). HOY algorithm [8] has $O(n^{d/2} \log n)$ time complexity, although it was initially thought to have $O(n^{d/2})$ time complexity [10] due to a gap in the analysis [8]. It is based on an algorithm for Klee's measure problem by Overmars and Yap and uses a streaming variant of an orthogonal partition tree as the underlying data structure. Only

²The algorithm was proposed in [60] and was later named and studied in more detail in Reference [87]. Note that Reference [87] also assign independent authorship of such an algorithm to Zitzler and provide a reference to source code (<ftp://ftp.tik.ee.ethz.ch/pub/people/zitzler/hypervol.c>).

$O(n)$ space complexity is required by constructing the referred tree on-the-fly instead of storing it completely as in the classical variant, which would require $O(n^{d/2})$ space complexity.

HOY is based on the Spatial Divide-and-Conquer paradigm (see Section 3.1.3). It measures the hypervolume dominated by a given point set $A \subset \mathbb{R}^d$ inside the region bounded by a lower and an upper reference point, ℓ and u , respectively. The set A is initialized as the set of input points, u to the given reference point and ℓ to the component-wise minimum of all input points. HOY recursively partitions in two parts the considered region, firstly using dimension one. Then, HOY checks if dimension i satisfies certain criteria for partitioning, otherwise, it checks dimension $i + 1$ and so on. The recursion base case is achieved when the (d -dimensional) dominated region of the current partition forms a trellis. Moreover, before HOY is used, points are previously sorted according to dimension d . This allows to shrink the space partition considered in each recursive call by visiting points in A in ascending order of dimension d until a point q that dominates the $(d - 1)$ projection of the space partition is found. In such case, the hypervolume of the space region between q_d and u_d is computed right away and u_d can be set to q_d .

[A5] Walking Fish Group algorithm. WFG algorithm [85] is currently one of the fastest for computing the hypervolume indicator in many dimensions, particularly for $d > 7$ (see the experimental results in Reference [64]), even though it is not asymptotically the fastest. WFG was initially reported to have $O(2^{n+1})$ time complexity [85] but Lacour et al. [64] recently tightened this upper bound to $O(n^{d-1})$ and presented a lower bound of $\Omega(n^{d/2} \log n)$.

WFG is an algorithm mainly based on the bounding technique and on the inclusion-exclusion principle, and is further optimized by integrating ideas of dimension sweep and objective reordering. WFG works as follows, given the point set $X \subset \mathbb{R}^d$ and a reference point $r \in \mathbb{R}^d$. Points in X are sorted and visited in ascending order of dimension d . For each point $p \in X$ visited, the $(d - 1)$ -dimensional contribution of p to the set of already visited points, S (i.e., $H(p^*, S^*)$), is then computed and multiplied by the difference $(r_d - p_d)$. The hypervolume of S is the sum of all such multiplications. The contribution $H(p^*, S^*)$ is computed by subtracting to the hypervolume of $\{p^*\}$ the hypervolume of the set S^* but bounded by p^* , i.e., $H(p^*, S^*) = H(p^*, J) = H(\{p^*\}) - H(J)$, where J is the set obtained with the bounding technique (see Section 3.2.1). In its turn, $H(J)$ is computed recursively with WFG. In practice, WFG alternates between ONECONTRIBUTION and HYPERVOLUME problems.

The bounding technique in combination with dimension sweep allows many points to become dominated and, thus, it plays an important role in reducing the required computational effort. This places WFG among the fastest algorithms for many dimensions ($d > 4$).

[A6] Chan's algorithm (Chan). Chan's algorithm [33] has $O(n^{d/3} \text{polylog } n)$ time complexity, which is currently the best time complexity to compute HYPERVOLUME for $d \geq 4$. Chan's paper proposes an algorithm for the general case of Klee's Measure problem and then derives an algorithm for the special case of the hypervolume indicator (referred as the union of arbitrary orthants). Chan's algorithm is a spatial divide-and-conquer algorithm combined with a procedure to simplify partitions that is repeated in every few levels of cutting, reducing the number of boxes contained in a partition. One of the main differences to HOY (see [A4]) is the method used for selecting the dimension to be used for partitioning, which changes at every level of recursion: first, it uses dimension one, then dimension two, and so on. Despite its good time complexity, no implementation is found available online.

[A7] Quick Hypervolume (QHV and QHV-II). QHV [72] is an algorithm for the general case of HYPERVOLUME problem that is based on quicksort. This algorithm can be viewed as a type of spatial divide-and-conquer but instead of dividing the hypervolume in two, as discussed in the

example given in Section 3.1.3, it splits the region into 2^d regions. To compute `HYPERVOLUME` for a point set $S \subset \mathbb{R}^d$, first a point $p \in S$ is selected as a pivot. The point in S with the greatest contribution to the empty set is selected for this purpose. The pivot splits the d -dimensional space into $O(2^d)$ hyperoctants sharing a corner at p and p is discarded. The two hyperoctants referring to the regions that contain the points dominating p or the points that p dominates are discarded. QHV is called recursively for each of the remaining hyperoctants (still partially dominated by S) with the points inside that region and the projection on the hyperplanes delimiting the hyperoctant of the points that partially dominate it. QHV includes a subroutine to discard dominated points from each hyperoctant. The hypervolume of S is the sum of the hypervolume returned from the recursive calls plus the contribution of p to the empty set. In base cases with up to 10 points, a simple algorithm as HSO or based in the Inclusion-Exclusion principle is used.

In the worst case, QHV has $O(n^d n^2)$ -space complexity, and an initially reported time complexity of $O(n(d + \log^{n-2})2^{nd})$ [72], which was later tightened to $O(2^{d(n-1)})$ [56]. However, its performance depends on the characteristics of the dataset considered. For example, the authors showed that the time complexity on a dataset where points are uniformly distributed over a hypersphere is $O(dn^{1.1} \log^{d-2} n)$ for the average case and with high probability [72]. In practice, QHV was observed to be competitive with WFG. Although a parallel version of QHV exists [73], only the sequential version is taken into account in this article.

A modified version of QHV, called QHV-II, was recently proposed [56]. QHV-II uses a different partitioning scheme, where the pivot is used to split the d -dimensional space in d hypercuboids. Moreover, points whose projection became dominated, may not be immediately removed, because the dominance check of each new projection is performed only against the pivot. QHV-II has a $O(d^{n-1})$ time complexity. However, similarly to QHV, its performance depends on the dataset, for example, there is a given problem for which QHV-II has $\Theta(n \log^{d-1} n)$ time complexity [56]. Overall, Jaskiewicz [56] showed that QHV-II has better worst-case time complexity than QHV and empirically showed that QHV-II performs less operations. The author points out that, unlike the original implementation of QHV that uses low-level code optimizations and different algorithms for small subproblems, the implementation of QHV-II is simple, and conjectures that faster performance than QHV can be achieved by using the same low-level code optimizations.

[A8] Hypervolume Box Decomposition Algorithm (HBDA). Lacour et al. [64] proposed an algorithm to compute `HYPERVOLUME` for any number of dimensions. This algorithm, HBDA, computes the hypervolume indicator by partitioning the dominated region into $O(n^{\lfloor \frac{d}{2} \rfloor})$ axis-parallel boxes and adding up the corresponding hypervolumes. The partitioning results from computing all local upper bounds, where each box is associated to one local upper bound. The incremental version of the algorithm (HBDA-I) runs in $O(n^{\lfloor \frac{d}{2} \rfloor + 1})$ time and is characterized by computing a sequence of n `UPDATEHYPERVOLUME` problems, allowing input points to be processed in any order. This update consists of updating the set of local upper bounds and then recomputing the hypervolume indicator from the resulting box decomposition. Since the current box decomposition must be stored across iterations, $O(n^{\lfloor \frac{d}{2} \rfloor})$ space is required. By processing input points in ascending order of any given coordinate, the memory requirements are reduced to $O(n^{\lfloor \frac{d-1}{2} \rfloor})$, and the time complexity is improved to $O(n^{\lfloor \frac{d-1}{2} \rfloor + 1})$. HBDA-NI (the non-incremental version) has been shown to be competitive in $d \geq 4$ dimensions, but its memory requirements are a limiting factor for large d .

Note that HBDA-I can be easily adapted to recompute the hypervolume in $O(n^{\lfloor \frac{d}{2} \rfloor})$ time when the reference point is changed to a new location (and is still strongly dominated by every point in the input set), provided that the data structures are set up as in HBDA-I. It is enough to identify

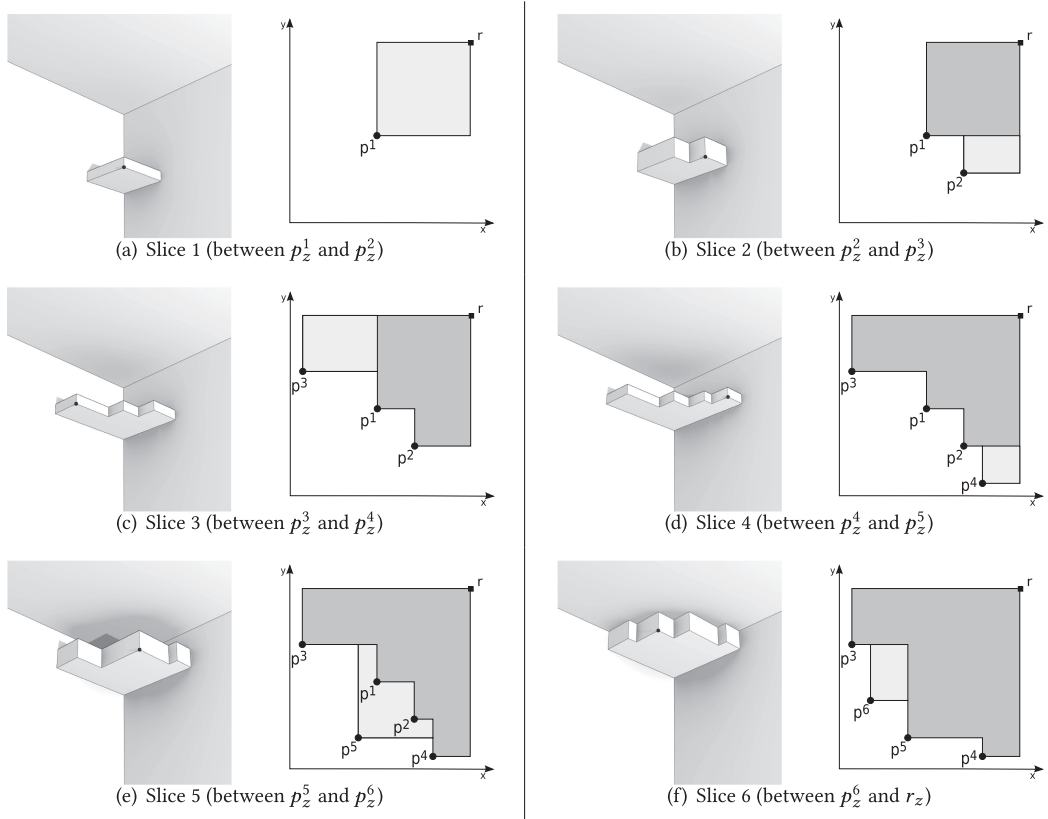


Fig. 12. The splitting of the volume in Figure 5(a) into 6 slices and the corresponding two-dimensional bases. The base area of the previous slice is depicted in dark gray in the 2-dimensional figures.

the local upper bounds with a coordinate of the old reference point, replace it with the coordinate of the new one, and then recompute the hypervolume from the updated set of local upper bounds.

[A9] Dimension-sweep algorithms for $d = 3$ (HV3D and HV3D⁺). HV3D [9] is a dimension-sweep algorithm for the $d = 3$ case of HYPERVOLUME problem with $\Theta(n \log n)$ time complexity and $O(n)$ space complexity. Given a point set $S = \{p^1, \dots, p^n\} \subset \mathbb{R}^3$, the volume dominated by S is divided into slices (see the example in Figure 6(a)) from bottom up. Each pair of figures in Figure 12 shows the three-dimensional representation of a slice and its two-dimensional base.

HV3D works by solving a sequence of 2-dimensional UPDATEHYPERVOLUME problems, as follows. Points in S are sorted and visited in ascending z -coordinate order. Each point $p \in S$ marks the beginning of a new slice, the base area of which is computed by updating the area of the base of the previous slice (if it exists). This is illustrated in Figure 12 where the darker gray region represents the base of the previous slice to be updated. To that end, the points visited so far whose projections on the (x, y) -plane are mutually nondominated are kept sorted in ascending order of the y coordinate using a height-balanced binary tree, T . In the example of Figure 12(d), T contains $\{p^4, p^2, p^1, p^3\}$ after computing the fourth slice. For each $p \in S$, the outer delimiter to the right ($q = p^4$ in the example of slice 5 where $p = p^5$) is determined in $O(\log n)$ steps by searching T . Then, the contribution of p^* to T^* is computed by visiting the successors of q in T in ascending order of y until the outer delimiter to the left is found (in the example, p^3). Such a contribution is

computed in a structure-destructive manner as explained in Section 3.2.4. In this case, each point in T weakly dominated by p on the (x, y) -plane is removed from T in $O(\log n)$, and p is added to T next to its outer delimiters in $O(\log n)$ as well. The base area of the new slice is computed by summing the contribution of p^* and the base area of the previous slice, and the volume of the slice is computed by multiplying its base area by its height. In the example of slice 5, after computing its volume, T contains the points delimiting its base, i.e., T contains $\{p^4, p^5, p^3\}$.

In the above, each point in S is visited twice: once when it is added to T and again when it is removed from T . Since all of the corresponding operations are performed in $O(\log n)$ time, the algorithm has amortized $O(n \log n)$ time complexity.

A modified version of HV3D that supports linear-time updates was recently proposed [50]. It is called HV3D⁺, it has the same time and space complexities but it is faster than HV3D [50]. The key difference between the two is that in HV3D⁺ the data structure setup is performed as a preprocessing step, in $O(n \log n)$ time, previous to the computation step, that is performed in $O(n)$ time, and not altogether as in HV3D. This data structure keeps track, for each point $p \in S$, the successor point according to coordinate z , and the outer delimiters of p at $z = p_z$. By performing the same sweep and the same operations in the binary tree T as in HV3D, and recalling the two neighbour points in the tree next to which each point is inserted to (the two outer delimiters of p), allow HV3D⁺ to then subsequently perform the same sweep more efficiently, in $O(n)$ time. The key idea is to avoid having to query (again) the tree T to find the two points next to which each point is going to be inserted to in T , when the same sweep is repeated. Consequently, the binary tree can be replaced by a linked list, and all $O(\log n)$ -time operations become $O(1)$ time instead.

An advantage of HV3D⁺ is that when a point is added/removed to the set (UPDATEHYPERVOLUME problem), the data structure can be updated in linear time and the hypervolume is then recomputed in linear time as well. This update procedure is called HV3D⁺-R. There is an alternative update procedure, called HV3D⁺-U, that only computes the contribution of the point being added/removed (see [B6]). As HV3D⁺-U performs less computations than HV3D⁺-R (it skips a few steps), the former computes UPDATEHYPERVOLUME problem faster. In practice, it can be up to three times faster than HV3D⁺-R [50]. An advantage of the HV3D⁺-R version is that it can be used to recompute the hypervolume in linear time after changing the reference point.

In the decremental scenario both update versions, HV3D⁺-R and HV3D⁺-U, assume that S is a nondominated point set but admit that points in S may be dominated by p in the incremental case. Admitting dominated points in the latter case is only possible because, unlike HV3D, the contribution of a point to the base of a slice is computed in a structure-preserving way. This is the key factor that makes possible the extension of HV3D⁺ to the all contributions problem (see [B3]).

[A10] Dimension-sweep algorithms for $d = 4$ (HV4D and HV4D⁺). HV4D [51] and HV4D⁺ [50] are amortized $O(n^2)$ -time and $O(n)$ -space algorithms for the particular case of $d = 4$ of the HYPERVOLUME problem. Although they have worse time complexity than Chan's algorithm (see [A6]), they are currently the fastest ones among the algorithms with available implementations.

HV4D algorithm is an extension of HV3D to four dimensions where a sequence of three-dimensional UPDATEHYPERVOLUME problems is solved via the corresponding ONECONTRIBUTION problems using similar techniques to those in the EF algorithm (see [B3]). Points in the input set $S \subset \mathbb{R}^4$ are visited in ascending order of the last coordinate, partitioning the dominated region into four-dimensional slices. For each $p \in S$, the base volume of the new slice is computed by updating the volume of the base of the previous slice with the contribution of p^* to the projection on (x, y, z) -space of the points visited so far.

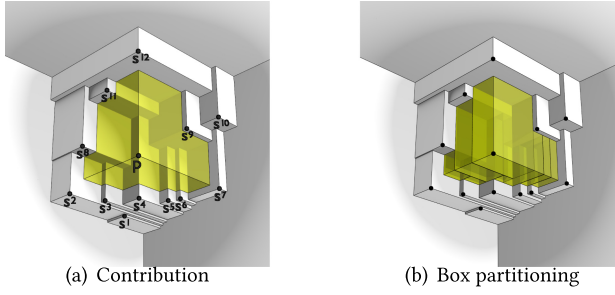


Fig. 13. Example of a contribution, $H(p^*, L^*)$, and of its division in boxes, where $L^* = \{s^1, \dots, s^{12}\}$.

For that purpose, the points visited so far whose projections are nondominated are stored in a data structure, L . The contribution of each $p^* \in S^*$ to L^* is computed using a procedure that partitions the contribution in boxes (see example in Figure 13) and takes linear time provided that points in L are sorted in two lists in ascending order of the y and z coordinates, respectively, and, provided that $L^* \cup \{p^*\}$ is a nondominated point set. However, even though S is a nondominated point set, S^* may not be and consequently, p^* may dominate some points in L^* . To fulfill the last requirement, the contribution of p^* to L^* is computed in a structure-destructive manner by removing the points dominated by p^* one by one as explained in Section 3.2.4.

Since a three-dimensional contribution is computed at most twice for each input point, once when it is added to L and once in case it is removed from L , then $O(n)$ calls to the procedure to compute a three-dimensional contribution are performed and consequently, the time complexity of HV4D amortizes to $O(n^2)$.

HV4D⁺ is a modified and faster [50] version of HV4D that uses the $O(n)$ -time HV3D⁺-U algorithm (see [B6]) to compute the three-dimensional contribution. Because HV3D⁺-U computes the contribution in a structure-preserving way, i.e., dominated points do not have to be explicitly removed previously, only n calls to HV3D⁺-U are performed.

A straightforward way to generalize these approaches is to use them as base cases and apply the same sweeping mechanism for each additional dimension, resembling HSO-based approaches, with an additional $O(n)$ factor per dimension. The i -th dimensional subproblem, where $i > 4$, is solved as a sequence of $(i-1)$ -dimensional ONECONTRIBUTION or UPDATEHYPERVOLUME problems by visiting points in ascending order of dimension i . Note that sorting can be done as a preprocessing step and be avoided at each dimension using FPL-like data structures.

4.2 Remarks

Table 1 summarizes the algorithms described in Section 4.1. It indicates for how many dimensions they can be used for (d) and for which they are recommended (d') based on their runtime performance and/or time complexity. Moreover, it indicates whether an implementation of such algorithms is available online and which are the paradigms/techniques used (see Sections 3.1 and 3.2). The bottom part of Table 1 summarizes the best algorithms regarding time complexity and/or runtime performance (based on the experimental results presented in References [50, 64, 72]).

HV3D⁺ is the recommended algorithm for $d = 3$, since it is optimal and the fastest in practice. For $d \geq 4$, Chan's algorithms is recommended since it has the best time complexity, but, because no implementation is available and there is no information on how it performs in practice, other algorithms are recommended as alternatives. For $d = 4$, HV4D⁺ is the fastest one. For $d = 5, 6$, HBDA-NI [64] held the best runtimes while for $d \geq 7$ it is not always clear which is the fastest one among WFG, QHV/QHV-II, and HBDA-NI, particularly because their ranking seems to be

Table 1. Algorithms for the HYPERVOLUME Problem

Algorithm	d	d'	Time complexity	Available	Characteristics
LebMeasure [A1]	≥ 2	—	$O(n^d)$	?	—
HSO [A2]	≥ 2	—	$O(n^{d-1})$	Yes	DS
HOY [A4]	≥ 2	—	$O(n^{d/2} \log n)$	Yes	SDC, DS
FPL [A3]	≥ 2	2	$O(n^{d-2} \log n)$	Yes	DS
HV3D ⁺ [A9]	3	3	$\Theta(n \log n)$	Yes	DS
HV4D ⁺ [A10]	4	4	$O(n^2)$	Yes	DS
Chan [A6]	≥ 4	≥ 4	$O(n^{d/3} \text{polylog } n)$	No	SDC
HBDA-NI [A8]	≥ 2	5, 6	$O(n^{\lfloor \frac{d}{2} \rfloor})$	Yes	LUBs
WFG [A5]	≥ 2	≥ 7	$\Omega(n^{d/2} \log n)$	Yes	IE, B, DS
QHV [A7]	≥ 2	≥ 7	$O(2^{d(n-1)})$	Yes	SDC
QHV-II [A7]	≥ 2	≥ 7	$O(d^{n-1})$	Yes	SDC

DS, Dimension sweep; IE, Inclusion-Exclusion; B, Bounding Technique; SDC, Spatial Divide-and-Conquer; LUBs, Local Upper Bounds.

strongly dependent of the input data, see References [56, 64, 72]. Since HBDA-NI only outperformed the other two in the experiments in Reference [64] in a specific dataset that is particularly difficult for WFG and because its memory requirements grow exponentially with d , HBDA-NI is not recommended for $d \geq 7$. In such cases, WFG and QHV/QHV-II are preferable. In particular, the performance of QHV was similar, and sometimes slightly better, than QHV-II in all datasets presented in Reference [56], except for the hard dataset proposed in Reference [64]. However, recall that the implementation of QHV-II can be improve by, for example, using the fast computations for small subproblems used in QHV. Compared to WFG, QHV was faster for random and discontinuous datasets [72] and showed similar performance for convex spherical dataset. For spherical concave, linear, degenerate, and the hard datasets, WFG was faster than QHV [64], being several orders of magnitude faster for the degenerate case.

The performance of the algorithms can greatly change depending on the considered dataset. Though this may not be problematic in low-dimensional cases as the existing algorithm are very efficient, the same cannot be said for high-dimensional cases where, for example, computing the hypervolume for 1,000 points in $d = 10$ in a given dataset may take less than a second, and for others it may take more than an hour (see Reference [72]). An alternative for the latter cases are approximation algorithms (e.g., References [21, 41, 57]). Note that some of these algorithms are, in fact, only advantageous for high dimensional cases, because for a low number of dimensions (e.g., $d < 10$) computing the exact hypervolume may be faster than approximating it (see References [50, 72]).

Note that HBDA-I and (one of) the update variants of HV3D⁺, HV3D⁺-R (see [A9]), can be used to efficiently update the hypervolume under reference-point changes, in $O(n^{\lfloor d/2 \rfloor})$ and $O(n)$ time, respectively. Both algorithms can also be used for the UPDATEHYPERVOLUME problem, although the former only for the incremental scenario. However, the hypervolume indicator can be updated faster with HV3D⁺-U (see [B6]) to solve the ONECONTRIBUTION problem for $d = 3$ [50], and an adapted version of WFG as in Reference [64] for $d > 3$.

Note that, most of the currently fastest algorithms available (HV3D⁺, HV4D⁺, and WFG) are all dimension-sweep based. Each one solves a sequence of subproblems smaller in the number of dimensions and in a way that avoids recomputing everything from scratch and/or that tries to reduce the problem size. For example, HV3D⁺ and HV4D⁺ solve HYPERVOLUME by iterating over UPDATEHYPERVOLUME (incremental scenario) while WFG alternates between ONECONTRIBUTION

and HYPERVOLUME problems and takes advantage of the bounding technique. By doing so, they all reduced the computational costs required when compared to HSO.

5 HYPERVOLUME CONTRIBUTIONS

Hypervolume-based selection is typically related to the ALLCONTRIBUTIONS and/or the HSSP problems, while the HYPERVOLUME problem is more related to the performance evaluation of EMOAs. The computation of hypervolume contributions is frequently required either directly by the EMOA selection method (e.g., in the special case of $k = n - 1$ of HSSP, or for ranking), or indirectly, in the inner steps of hypervolume-related algorithms (e.g., in algorithms to approximate the HSSP). Although algorithms for the HYPERVOLUME problem can also be used to compute contributions by solving a sequence of HYPERVOLUME problems, it is typically more advantageous to use algorithms particular to such problems. Thus, the algorithms in Section 4 should be used only as last resort, when there is no problem-specific alternative.

This section focuses on algorithms for problems related to the exact computation of hypervolume contributions, in particular: ALLCONTRIBUTIONS, ONECONTRIBUTION, LEASTCONTRIBUTOR, UPDATEALLCONTRIBUTIONS, and the UPDATEALLCONTRIBUTIONS2 problem. Only the state-of-the-art algorithms are described. Those (non-competitive) algorithms purely based on HSO (e.g., Reference [89]) and approximation algorithms (e.g., References [5, 22]) were left out.

5.1 Algorithms for ALLCONTRIBUTIONS Problem

[B1] Bringmann and Friedrich algorithm (BF1). Bringmann and Friedrich [24] proposed an algorithm for the HSSPCOMPLEMENT problem, here referred as BF (see [C1]). Given a point set $S \subset \mathbb{R}^d$ of n points and a subset size k , the algorithm computes the contribution to S of every subset of $n - k$ points. In the particular case of $k = n - 1$, the algorithm computes the contribution of every subset of size 1, i.e., the ALLCONTRIBUTIONS problem. For this particular case, the algorithm has $O(n^{d/2} \log n)$ time complexity and will be here referred to as BF1.

[B2] Exclusive (Contribution) Quick Hypervolume (exQHV). The authors of QHV ([A7]) extended the algorithm for the ALLCONTRIBUTIONS problem in any number of dimensions [73]. The main differences to QHV are (1) the need to store all contributions instead of a single value, (2) the point used as pivot, and (3) the exclusion of dominated points. In the case of QHV, the point p with greatest contribution to the empty set is the pivot. Because the contribution of p also has to be computed, the authors decided not to use p as pivot, but to use instead the point p' with greatest contribution to the empty set among the points obtained from the coordinate-wise maximum between p and each point in the hyperoctant. In QHV, all dominated points in an hyperoctant are discarded. However, in exQHV, points dominated by a single point are needed for computing the contribution of the point dominating it and, therefore, only the points dominated by two or more points are excluded from each hyperoctant. As in QHV, an algorithm such as HSO or based on the Inclusion-Exclusion principle is used for small cases with up to 10 points. Such algorithms were also adapted to compute contributions [73].

[B3] Dimension-sweep algorithms for $d = 2, 3$ (EF2D, EF, and HVC3D). Emmerich and Fonseca [39] proposed a $\Theta(n \log n)$ algorithm for the $d = 2$ case of the ALLCONTRIBUTIONS problem, here referred to as EF2D, that computes all (box-shaped) contributions in linear time after sorting the input set. EF [39] is a dimension-sweep algorithm for the $d = 3$ case that has $\Theta(n \log n)$ time complexity and $O(n)$ space complexity. This algorithm extends HV3D to the computation of the contributions of all points in a nondominated point set $S \subset \mathbb{R}^3$. As in HV3D, EF visits all points in S in ascending order of z -coordinate while maintaining a balanced binary tree T . This tree plays the

same role as in HV3D, that is, to maintain the visited points (nondominated on the (x, y) -plane) that delimit the base of the current slice. These are now also the points whose contributions are being computed, and when the projection of a point becomes dominated and the point is removed from T , it also means that its contribution is fully computed. Therefore, in addition to the data structure used in HV3D, for each point $p \in T$, a partition of the contribution of p^* to T^* is stored in a doubly-linked list of boxes and the contribution of p is computed with the update of such a list, which is updated along with the update of T . A total of $O(n)$ boxes are created, each operation on a box has a $O(1)$ cost, and all operations on boxes amortize to $O(n)$. The time complexity of EF is dominated by the operations in T , with an overall amortized $O(n \log n)$ time complexity, as HV3D.

HVC3D [50] is a $O(n \log n)$ time algorithm that is an extension of HV3D⁺ (see [A9]) to the $d = 3$ case of the ALLCONTRIBUTIONS problem. HVC3D works in a similar way to EF but allows linear-time updates (see [B10]). By using the data structure of HV3D⁺ that takes $O(n \log n)$ time in the pre-processing step, the tree T is replaced by a doubly-linked list, which allows to compute all contributions in amortized $O(n)$ time. Moreover, instead of a list of boxes, each point p in T (now stored in a linked list) has an associated sorted doubly-linked list with the delimiters of its contribution on the (x, y) -plane. Such a list of delimiters is updated in a structure-preserving way as new points from S are visited, even if a point dominated by p is visited. Moreover, unlike EF, the fact that HVC3D can be extended to efficiently update contributions, even in the presence of dominated points, allows it to be further used in an algorithm for the $d = 4$ case (HVC4D).

[B4] Dimension-sweep algorithm for $d = 4$ (HVC4D). HVC4D [50] is a dimension-sweep algorithm for the $d = 4$ case of the ALLCONTRIBUTIONS problem, and it has $O(n^2)$ -time and $O(n)$ -space complexity. It is currently the asymptotically and the empirically fastest algorithm for this case [50]. It can be viewed as an extension of HV4D⁺ (see [A10]) for the all contributions case. It solves a sequence of 3-dimensional cases of the incremental scenario of UPDATEALLCONTRIBUTIONS problem in linear time using HVC3D-U (see [B10]). Points in the input set $S \subset \mathbb{R}^4$ are visited in ascending order of the last coordinate, partitioning the dominated region into 4-dimensional slices. For each new visited point $p \in S$, the three-dimensional contributions to the projection on the (x, y, z) -space of p and of the points delimiting the base of the previous slice are updated, given that p is added to the new slice. The four-dimensional contributions to the new slice of such points are computed by multiplying their three-dimensional contribution by the height of the slice.

5.2 Algorithms for ONECONTRIBUTION Problem

[B5] Incremental HSO (IHSO). IHSO [14, 15] is an algorithm for computing the contribution of a single point, $p \in \mathbb{R}^d$, to a set of points $X \subset \mathbb{R}^d$, i.e., for the ONECONTRIBUTION problem. It is based on HSO, but only the contribution of p to X is split into slices whose hypervolumes are summed up at the end. Points that are not delimiters of the contribution of p to X are ignored. Additionally, an objective reordering heuristic is used to choose a good order to process the objectives. IHSO works even if some points in X are dominated by p .

There is an improved version of IHSO that uses FPL's data structure (see [A3]). It was proposed as part of IIHSO [17], which is an algorithm for the HYPERVOLUME problem that uses IHSO to iteratively solve ONECONTRIBUTION problems.

[B6] HV3D⁺ update version (HV3D⁺-U). HV3D⁺-U [50] is a version of HV3D⁺ (see [A9]) to compute the contribution of a point $p \in \mathbb{R}^3$ to a set $S \subset \mathbb{R}^3$ in linear time, even if p dominates points in S , provided that the data structure is already set up as in HV3D⁺. HV3D⁺-U performs the same computations as HV3D⁺ but restricted to the region dominated by p , i.e., only the contribution of p

to S is partitioned in slices. Thus, it performs the same sweep and management of the data structure, but as only the base area of the slices where p^* has contribution have to be computed/updated, it skips the computation of the volume of slices below $z = p_z$, and stops as soon as a point in S that dominates p on the (x, y) -plane is visited.

HV3D⁺-U has resemblances to the procedure used by HV4D to compute the $d = 3$ case of ONECONTRIBUTOR problem in linear time (see [A10]). Apart from how they represent the contribution of p , the former through its delimiters and the latter through boxes, both require the data structures to be previously set up. Moreover, they work in a similar way when $\{p\} \cup S$ is a nondominated point set, however, this is actually a requirement of HV4D but not of HV3D⁺-U.

5.3 Algorithms for LEASTCONTRIBUTOR

[B7] Incremental HSO* least contributor algorithm (IHSO*). IHSO* [14] is an algorithm based on IHSO (see [B5]) for the LEASTCONTRIBUTOR problem. In the search for the least contributor, IHSO* tries not to fully compute all contributions. Of all schemes tested in Reference [14], the best approach to avoid full computations was to use an objective reordering heuristic combined with **Best-First-Queueing (BFQ)**. At each step of BFQ, the point with the currently lowest (partially) computed contribution is picked and a “bit” more of its contribution is computed. A parameter of BFQ defines how much a “bit” more is. This process continues until a point with its contribution fully computed is picked and this is the least contributor. However, the notion of a “bit of contribution” has to be reasonably defined, i.e., if the considered granularity is too large, then all contributions are computed, if too small, then an excessive number of iterations is required. The time complexity of IHSO* is not reported. IHSO* authors also proposed an algorithm to update the least contributor under single-point changes [12], i.e., to identify the (new) least contributor after adding or removing a point.

[B8] Incremental WFG least contributor algorithm (IWFG). IWFG [35, 84] is an algorithm for the computation of the LEASTCONTRIBUTOR and is the result of a combination between WFG (see [A5]) and IHSO* (see [B7]). It uses the best-first queuing mechanism of IHSO* to gradually update contributions until the least contributor is found. Therefore, in IWFG, the contribution of each point is divided into slices and the first slice associated to each point is computed using WFG. Then, repeatedly, the point with the lowest (partially) computed contribution is picked and its next slice is computed also using WFG. The algorithm stops when the selected point is one whose contribution is fully computed (is the least contributor). Cox and While [35] proposed an improved version of IWFG that uses a new slicing scheme and reordering of objectives for each point, which led to better runtimes and to outperform IHSO*. The time complexity of IWFG is not reported.

5.4 Algorithms for UPDATEALLCONTRIBUTIONS

[B9] Hupkens and Emmerich update algorithm for $d = 2$ (UHVC2D). Hupkens and Emmerich [53] proposed an algorithm, here called UHVC2D, to update the contributions of every point in a set $S \subset \mathbb{R}^2$ to the set S itself under single-point changes to S (incremental and decremental cases of UPDATEALLCONTRIBUTIONS), in the $d = 2$ case. This algorithm has $O(n)$ space complexity and $O(\log n)$ time complexity provided that S is a nondominated set and is previously stored sorted along a coordinate in a balanced binary tree. Igel et al. [54] had previously proposed an update method very similar to UHVC2D. However, it was proposed as a part of a procedure for ranking solutions and only the decremental scenario was considered.

Note that there are at most two delimiters of the contribution of a point $p \in S$ to a nondominated set $S \subset \mathbb{R}^2$ and each point in S only delimits the contribution of its delimiters. Therefore, UHVC2D does a $O(\log n)$ -time search in the tree that stores S to find the delimiters of the contribution of the point p being added/removed. The contribution of p and each delimiter is then updated in $O(1)$ time. The incremental case has an extra step to remove points in S dominated by the new point before the contribution update. As the tree insertion/deletions are performed in $O(\log n)$, UHVC2D takes amortized $O(\log n)$ time per point. Moreover, note that the algorithm could be easily applied to the UPDATEHYPERVOLUME problem for $d = 2$.

[B10] Guerreiro and Fonseca algorithms for $d = 3$ (HVC3D-R and HVC3D-U). Guerreiro and Fonseca [50] proposed two algorithms based on the HVC3D algorithm (see [B3]) to solve the $d = 3$ case of UPDATEALLCONTRIBUTIONS problem, namely HVC3D-R and HVC3D-U. Both algorithms have $O(n)$ -time and $O(n)$ -space complexity provided that for a point set $S \subset \mathbb{R}^3$, the data structures are already set up (as in HVC3D), and the corresponding contributions are known (only needed by HVC3D-U). HVC3D-R and HVC3D-U also correspond to extension of HV3D⁺-R and HV3D⁺-U algorithms (see [A9] and [B6]), respectively, to the case of all contributions. Hence, HVC3D-R consists of adding/removing a point to S , updating the data structure in linear time similarly to HV3D⁺-R, and then recomputing all contributions as HVC3D, in linear time. HVC3D-R can also be used to recompute all contributions in linear time after the reference point is changed. HVC3D-U works like HVC3D-R, the main difference being that it only computes the (part of the) contributions that are inside the region dominated by the point $p \in \mathbb{R}^3$ being added/removed, i.e., the joint contributions. Similarly to HV3D⁺-U, this allows to skip some steps that HVC3D-R has to perform. Namely, the contributions in the slices below $z = p_z$ are not computed, and the algorithm stops as soon as a second point in S that dominates the projection of p on the (x, y) -plane is found. The point set $S \cup \{p\}$ is assumed to be nondominated only for the decremental scenario, while this is not required for the incremental case. HVC3D-U can be up to two times faster than HVC3D-R [50].

5.5 Algorithms for UPDATEALLCONTRIBUTIONS2

[B11] Guerreiro et al. algorithms for $d = 2, 3$ (gHSS2D-u and gHSS3D-u). Integrated in the (incremental) greedy algorithms gHSS2D and gHSS3D to approximate the HSSP (see Section 6.2), Guerreiro et al. [52] proposed update procedures for the $d = 2$ and the $d = 3$ cases of UPDATEALLCONTRIBUTIONS2 problem, respectively. Recall that in such a case, given the sets of points $S, R \subset \mathbb{R}^d$, the goal is to update the contributions to R of the points in S when a point $p \in \mathbb{R}^d$ is added/removed from R . The update procedures in gHSS2D and gHSS3D work for both incremental and decremental scenarios provided that $S \cup R$ is a nondominated point set. Let us call these procedures gHSS2D-u and gHSS3D-u, respectively. The gHSS2D-u algorithm updates such contributions in $O(|S| + |R|)$ time provided that the data structure is already set up, i.e., points are previously sorted and a flag identifies to each set, R or S , each point belongs to. By visiting points in order once, each box-shaped contribution is updated in constant time.

The gHSS3D-u is a dimension sweep algorithm that uses objective reordering. Assuming that $S \cup R \cup \{p\}$ is a nondominated point set, and that $S \cup R \setminus \{p\}$ is previously sorted in linked lists according to all dimensions, gHSS3D-u works as follows. It first considers the objective order (x, y, z) , updates the contributions of points in S that are in the octact containing the points dominated by p on the (x, y) -plane (and have lower z -coordinate than p), and then those in the octact containing the points that dominate p on the (x, y) -plane (and have higher z -coordinate than p). Then it repeats this procedure for the other points in S considering the objective orders (y, z, x) , and

Table 2. Algorithms for Computing Hypervolume Contributions-related Problems

Problem	Algorithm	d	Time complexity	Available
ALLCONTRIBUTIONS	BF1 [B1]	≥ 2	$O(n^{d/2} \log n)$	No
	EF2D [B3]	2	$O(n \log n)$	Yes*
	HVC3D [B3]	3	$O(n \log n)$	Yes
	HVC4D [B4]	4	$O(n^2)$	Yes
	exQHV [B2]	≥ 2	—	Yes
	WFG-c [A5]	≥ 2	$\Omega(n^{d/2} \log n)$	Yes*
ONECONTRIBUTION	IHSO [B5]	≥ 2	—	No
	HV3D ⁺ -U [B6]	3	$O(n)$	Yes
	WFG-c [A5]	≥ 2	$\Omega(n^{d/2} \log n)$	Yes*
LEASTCONTRIBUTOR	IHSO* [B7]	≥ 2	—	No
	IWFG [B8]	≥ 2	—	Yes
UPDATEALLCONTRIBUTIONS	UHVC2D [B9]	2	$O(\log n)$	Yes*
	HVC3D-U [B10]	3	$O(n)$	Yes
UPDATEALLCONTRIBUTIONS2	gHSS2D-u [B11]	2	$O(n + m)$	Yes*
	gHSS3D-u [B11]	3	$O(nm)$	Yes*

(z, x, y) . A single call to gHSS3D-u has $O(|S| \cdot |R|)$ time complexity. When used sequentially for the incremental scenario (as in gHSS3D) the number of operations amortize to linear time per point added to R.

5.6 Remarks

Table 2 summarizes the information regarding the algorithms described in this section. It reports the best algorithms for each problem and the number of dimensions for which they can be used, the corresponding time complexities (if known) and indicates whether an implementation is available online. In the latter case, a “Yes*” means that an implementation is not exactly available but is easy to implement/obtain. In the case of UHVC2D and EF2D, the algorithms are easily implemented, and in the case of gHSS2D-u and gHSS3D-u, they can be extracted from the code of gHSS2D and gHSS3D, respectively. Although WFG was originally proposed for the HYPERVOLUME problem, with some (simple) modifications/adaptations it can be used for efficiently computing contributions, and such a version is here referred as WFG-c. Recall that WFG explicitly solves several ONECONTRIBUTION problems in its inner steps (see [A5]). The code for WFG available online can be easily modified to make use of that inner step to compute just the ONECONTRIBUTION and even to compute the ALLCONTRIBUTIONS problem (as it was the done in Reference [50] with version 1.11 of WFG for this latter case). This is more advantageous than iterating over the original WFG and use it to compute every contribution as the difference between two hypervolumes (see Definition 2.3).

The reported time complexities of UHVC2D, HV3D⁺-U, gHSS2D-u, and gHSS3D-u assume that the data structures were previously set up. In the case of UHVC2D, the reported time complexity represents the worst case if $\{p\} \cup S$ is a nondominated point set. Otherwise, it represents the amortized time complexity per point for a sequence of n calls, i.e., with n incremental updates to an initially empty set S . As proposed, gHSS2D-u and gHSS3D-u require $S \cup R \cup \{p\}$ to be a nondominated point set. The reported time complexity of gHSS3D-u amortizes to $O(|S| + |R|)$ per point in R, if used for a sequence of incremental cases, where points in R are added one by one.

For the ALLCONTRIBUTIONS problem, EF2D, HVC3D, and HVC4D are the recommended algorithms for $d = 2$, $d = 3$, and $d = 4$, respectively. For $d > 4$, as there is no available implementation of BF1, exQHV, and WFG-c are both recommended as alternatives.

The ONECONTRIBUTION problem is trivially solved for the $d = 2$ case in $O(n \log n)$. For the $d = 3$ case, the recommendation is to use HV3D⁺-U to compute it in linear time provided that the data structure is already set up, which can be done in $O(n \log n)$ time (see [A9]) if required. For $d = 4$, it is asymptotically faster to solve the HYPERVOLUME problem with HV4D⁺ than to use WFG-c. As there are no experimental comparisons, both are recommended. For $d > 4$, WFG-c is the recommended algorithm. Note, however, that any algorithm for the HYPERVOLUME problem can also be used to solve the ONECONTRIBUTION problem by taking advantage of the bounding technique (see Section 3.2.1) as WFG does (see [A5]), i.e., by computing $H(J)$ after a preprocessing step to determine the auxiliary point set J (see Definition 2.5).

IWFG is the only algorithm available online specific for the LEASTCONTRIBUTOR problem. However, in the experiments in Reference [50], using HVC3D and HVC4D to solve the ALLCONTRIBUTIONS problem for the $d = 3, 4$ cases, and then identifying the least contributor, was much faster (up to 456 times faster) than to use IWFG. The same is not observed for $d > 4$ in comparison to exQHV [73]. Therefore, for the $d = 3$ and the $d = 4$ cases of LEASTCONTRIBUTOR, HVC3D and HVC4D are recommended, respectively, and for $d > 5$, IWFG is recommended.

For the UPDATEALLCONTRIBUTIONS problem, the best is to use UHVC2D for $d = 2$, and HVC3D-U for $d = 3$, while for $d > 3$ the best alternative is to use the algorithms recommended for the ALLCONTRIBUTIONS problem. In the case of the update of contributions under reference-point changes, the $d = 2$ case is trivial, for the $d = 3$ case the recommendation is to use HVC3D-R (see [B3]), and for the $d > 3$ case, is to use the recommended algorithms for ALLCONTRIBUTIONS.

Due to the absence of dedicated algorithms for the ALLCONTRIBUTIONS2 problem and for the UPDATEALLCONTRIBUTIONS2 problem for $d > 3$, the recommendation is to iterate over algorithms for the ONECONTRIBUTION problem. For the $d = 2$ and $d = 3$ cases of UPDATEALLCONTRIBUTIONS2 problem, the recommendation is to use gHSS2D-u and gHSS3D-u, respectively.

Currently, there is no dedicated algorithm for the GREATESTCONTRIBUTOR2 problem. As mentioned in Section 2.3.1, an alternative is to solve it using an algorithm for the ALLCONTRIBUTIONS2 problem. In practice, since there is no dedicated algorithm for the latter problem as well, the alternative is to iteratively call an algorithm for the ONECONTRIBUTION problem.

6 HYPERVOLUME SUBSET SELECTION

Although the computation of the hypervolume indicator raised a lot of interest in the last decade and better algorithms have successively been proposed (see Section 4), there have been fewer contributions for the HSSP, which is a NP-hard problem [20]. There is currently no general algorithm for $d > 3$ that does not check every combination of k points. Only for $d = 2, 3$, or for small k or k close to n , are there a few algorithms that avoid that much computation. The existing algorithms to compute the HSSP exactly are briefly described next.

6.1 Exact Algorithms for the HSSP

Bringmann and Friedrich [24] proposed the first algorithm for the general d -dimensional case for the exact computation of the HSSP in 2010. This algorithm, which will be here referred as BF [C1], explicitly checks every combination of $(n - k)$ points to discard. As HOY (see [A4]), BF algorithm is an adaptation of Overmars and Yap's algorithm. BF has a $O(n^{d/2} \log n + n^{n-k})$ time complexity and $O(\min(n^{d/2}, n^{n-k}))$ -space complexity. Most of the remaining algorithms are focused on the particular cases of two and three dimensions, and are based on dynamic programming. In 2009, a $O(kn^2)$ -time dynamic programming algorithm was proposed for the $d = 2$

case [2], which will be referred here as DPHSS [C2]. A faster algorithm was proposed in 2014 by Bringmann et al. [26], and two other in 2016 by Kuhn et al. [63]. The algorithm by Bringmann et al. [26], HypSSP [C3], is also a dynamic programming algorithm that relies on modelling the contribution of each point to a set as a linear function of a reference point. The algorithm was initially shown to have $O(nk + n \log n)$ time complexity [26], but with slight modifications it can be tightened to $O(k(n-k) + n \log n)$ [46], leading to a new version called extreme2d [C4]. The algorithms by Kuhn et al. [63] rely on two alternative formulations of the HSSP/HSSPCOMPLEMENT: a formulation of the HSSPCOMPLEMENT as a k -link shortest path (k -LSP) problem [C5], for which the authors propose a dynamic programming algorithm that has $O(k(n-k) + n \log n)$ time complexity; and a formulation of the HSSP as an Integer Linear Programming (ILP) problem that relies in a partitioning of the dominated region in $O(n^d)$ boxes, and which can be solved with any general-purpose ILP solver [C6]. These are interesting formulations even though the former works only for the two-dimensional case and the generalization of the latter to any number of dimensions [45, 62] has a number of constraints exponential in d . The space complexity of the above dynamic programming algorithms is $O(n)$ if only the hypervolume value of the optimal subset(s) is required, and have either $O(nk)$ (DPHSS, HypSSP, extreme2d) or $O(n^2)$ space complexity (k -LSP) if such a subset is also needed.

More recently, Bringmann et al. [20] proposed a $O(n^{\sqrt{k}})$ time dynamic programming algorithm for $d = 3$, which exploits the fact that the boundary of the volume of a point set S can be described by a planar graph with $O(|S|)$ vertices, and the use of separators to split the problem. This algorithm will be referred here as BCE3D [C7]. For the general d -dimensional case, Gomes et al. [45] proposed a **branch-and-bound algorithm (B&B)** [C8], and a few methods to compute upper bounds. These upper bounds rely on hypervolume contributions (ALLCONTRIBUTIONS and ALLCONTRIBUTIONS2 problems), and consequently, the runtime efficiency of the branch-and-bound algorithm relies on the efficiency of the update algorithms used for the UPDATEALLCONTRIBUTIONS and UPDATEALLCONTRIBUTIONS2 problems. The most recent paper on HSSP algorithms focused on a different perspective, Groz and Maniu [46] proposed algorithms for the cases where either k or $n - k$ is small, which avoid enumerating all subsets by relying on data structures for extreme-point queries, range trees, hash maps, and other techniques. In particular, the authors propose a $O(n \log^d n)$ algorithm for $k = 2$. For $d = 3$, they propose algorithms with the following time complexities: $O(n \log^2 n)$ for $k = 2$, $O(n \log n)$ for $k = n - 2$, $O(n^{3/2})$ for $k = n - 3$, and $O(n^{n-k-0.62 \lfloor \frac{n-k}{3} \rfloor})$ for the case where $n - k$ is small. These algorithms will be referred to as GM-sc [C9].

6.1.1 Remarks. Table 3 summarizes the existent algorithms for solving HSSP exactly, indicating the number of dimensions for which they can be used (d), and whether or not there is an implementation available online (“Available” column). Column d' indicates for which number of dimensions each algorithm is the most indicated/faster.

For the two-dimensional case, the currently fastest algorithms are extreme2d (the improved version of HypSSP [C3]) and k -LSP. It is worth emphasizing that, for a fixed n -point set and increasing k , the runtime of both algorithms (and of B&B) is progressively slower the closer k is from $n/2$ and then becomes faster as k approximates n [45, 46, 63].

For the three-dimensional case, there are currently five algorithms: BF, ILP, BCE3D, B&B, and GM-sc. Regarding time complexity, the best choice depends on n and k . For example, for small k , BCE3D and B&B should be the best choices, while for k close to n , BF, B&B, and GM-sc should be the best choices. Although B&B was shown to be faster than solving the ILP with scip [45], there are no other comparisons between exact algorithms for the $d = 3$ case, and implementations of only B&B and GM-sc can be found online.

Table 3. Exact Algorithms for the HSSP

Algorithm	d	d'	Time complexity	Available	Paradigm
BF [C1]	≥ 2	≥ 4	$O(n^{d/2} \log n + n^{n-k})$	No	SDC
DPHSS [C2]	2	—	$O(kn^2)$	?	DP
HypSSP [C3]	2	—	$O(kn + n \log n)$	Yes	DP
extreme2d [C4]	2	2	$O(k(n-k) + n \log n)$	Yes	DP
k -LSP [C5]	2	2	$O(k(n-k) + n \log n)$	Yes	digraph-based, DP
ILP [C6]	2,3	—	$\Omega(n^d)$	Yes*	
BCE3D [C7]	3	3	$n^{O(\sqrt{k})}$	No	DP
B&B [C8]	≥ 2	≥ 3	—	Yes	Branch-and-Bound
GM-sc [C9]	3	3	$O(n^{n-k-0.62 \lfloor \frac{n-k}{3} \rfloor})$	Yes	

For special cases of small k or small $n - k$, more efficient algorithms can be used. For the $k = 1$ case, the solution can be trivially computed in $O(n)$ time. For $k = n - 1$ the problem can be efficiently solved by solving the LEASTCONTRIBUTOR (or ALLCONTRIBUTIONS) problem (see the recommended algorithms in Section 5.6). For $k \geq n - 3$, there are the specific algorithms GM-sc, but B&B is recommended as well. For more general cases where k becomes closer to $\frac{n}{2}$ or for large n , the algorithms for $d \geq 3$ may not be practical. For example, in Reference [45], for $d = 3$, $k = \frac{n}{2}$ and n close to 100, B&B took 100 s for some datasets, and, in Reference [46], for $d = 3$, $k = n - 3$ and $n \geq 50,000$, GM-sc took more than 100 s. Therefore, the existing exact algorithms for $d \geq 3$ are recommended only for specific cases such as small n , or k close to either 1 or n . For any other cases in $d \geq 3$, if an approximation to the HSSP is enough, a greedy algorithm should be preferable.

For the $d > 3$ case, the B&B algorithm is likely the best alternative, otherwise it would be necessary to consider all possible subsets of size $n - k$ (with BF), which can be very time-consuming.

On the extension of the $d = 2$ algorithms to $d \geq 3$. In the study of optimal distributions for the three-dimensional case, there are up to n points that influence the optimal placement of a point and its contribution, i.e., there are up to n delimiters of the contribution of a point, while for the $d = 2$ case, there are only two [1]. Therefore, the choice of a point may depend on all other points. Consequently, that makes the extension of dynamic programming algorithms for the two-dimensional case to the three-dimensional case more difficult.

6.2 Approximation Algorithms for the HSSP

The greedy approximation of HSSP is an alternative to the computationally expensive exact algorithms, in particular for $d > 2$. Several greedy approaches were proposed in the literature. Most of these approaches are generic approaches that iterate over hypervolume-based problems and thus, implementing them with different algorithms for these problems leads to different instances of such approaches. This means that the time complexity and runtime of an instance of a generic greedy approach depends on the underlying hypervolume-based algorithm used to implement it, but the approximation result does not. In the following, let $X \subset \mathbb{R}^d$ be a nondominated point set such that $|X| = n$ and that an optimal subset, $S^{opt} \subseteq X$, of size $k \in \{1, \dots, n\}$ needs to be selected. Let $r \in \mathbb{R}^d$ be a reference point such that every point in X strongly dominates it.

There are four generic greedy approaches: the decremental greedy approach (called gHSSD) [D1], the incremental greedy approach (called gHSS) [D2], the local search-based approach

(here referred to as LS [D3]), and the **Global Simple Evolutionary Multiobjective Optimizer (GSEMO)** [D4].

The decremental greedy approach (also known as “Greedy Front Reduction” approach) [11, 13] consists of discarding $n - k$ points from X one at a time so as to maximize the hypervolume retained at each step. It may be solved as a sequence of **LEASTCONTRIBUTOR** problems, and more generally as a sequence of **UPDATEALLCONTRIBUTIONS** problems. The greedy incremental approach (also known as “Greedy Front Addition” approach) [13] consists of selecting k points from X one at a time so as to maximize the hypervolume gained at each step. It may be solved as a sequence of **GREATESTCONTRIBUTOR2** problems, and more generally as a sequence of **UPDATEALLCONTRIBUTIONS2** problems. In practice, it is not imperative to know the contributions to the set of selected points, S' , of all point in $X \setminus S'$ in order to select the one that contributes the most. That is the case of the $O(n \log n)$ algorithm proposed in Reference [46] for the $d = 2$ case, here referred to as **gHSSEx** [D5], that relies on a data structure for extreme point queries to query for the maximal contributor between two adjacent points in S' in logarithmic time. A lazy evaluation approach for the contribution update in **gHSS** also allows to identify the maximal contributor without having to know, at each iteration, the exact contribution of every point in $X \setminus S'$ to S' [34]. A point’s contribution is updated only if it holds the largest known (and possibly outdated) contribution. If after the update that point still has the largest contribution, then it is the greatest contributor. Both **gHSS** and **gHSSD** approaches provide an approximation ratio to the **HSSP**, even though the approximation ratio to the **HSSPCOMPLEMENT** with either approaches can be arbitrarily large (see Section 2.4). The **LS** approach [11, 13] is based on the premise that for large n and small values of k it may be advantageous to try out several sets of size k , i.e., perform several cheap computations instead of the more demanding ones of updating all contributions. In **LS**, a k -size subset S of X is initially selected randomly. Then, a small number of points are (randomly) replaced by others in $X \setminus S$ and the new subset is accepted if the hypervolume indicator has improved. Different schemes for replacing subsets of points were proposed in the literature (see References [6, 11, 13]). The last generic approach, **GSEMO** [44], consists of a multiobjective evolutionary algorithm that is expected to achieve a $(1 - 1/e)$ -approximation to the **HSSP** in $O(n^2(\log n + k))$ iterations. In **GSEMO**, each individual in the population encodes a subset S of X , which represents a solution to the **HSSP**. **GSEMO** considers the (maximization) of a bi-objective problem, where the first objective is the hypervolume indicator of the encoded set S if $|S| \leq k$ and is -1 otherwise, while the second objective is the number of points left out of S , i.e., $|X \setminus S|$. In each iteration of **GSEMO**, a new individual (obtained through parent selection and variation operators) is inserted in the population only if it is not weakly dominated.

In contrast to the above greedy approaches, Bringmann et al. [20] recently proposed an **efficient polynomial-time approximation scheme (EPTAS)** [D6]. The algorithm is based on Dynamic Programming, and approximates the **HSSP** for any constant number of dimensions, d , in $O(n\epsilon^{-d}(\log n + k + 2^{O(\epsilon^{-2} \log 1/\epsilon)^d}))$ time with an approximation guarantee of $O(1 - \epsilon)$ to the optimal solution. For any constant d and ϵ , the time complexity is $O(n(k + \log n))$.

6.2.1 Remarks. The most advantageous combination of each greedy approach with state-of-the-art hypervolume-based algorithms, for each number of dimensions, are shown in Table 4. This list includes the combinations that lead to algorithms with the best time complexity and includes the currently fastest alternatives than can be constructed from implementations found online. In the reported time complexity of **LS** and **GSEMO**, t stands for the number of evaluated subsets, of at most k points. In the case of **GSEMO**, t must be $O(n^2(\log n + k))$ to achieve a $(1 - 1/e)$ -approximation. Hypervolume-based algorithms with available implementations that are not available in

Table 4. Greedy Algorithms for the HSSP

Algorithm	Auxiliar Alg.	d	d'	Time complexity	Avail.
gHSSD [D1]	UHVC2D [B9]	2	2	$O((n-k) \log n)$	Yes*
	gHSSD3D (HVC3D-U) [B10]	3	3	$O((n-k)n + n \log n)$	Yes
	HVC4D [B4]	4	4	$O((n-k)n^2)$	Yes
	BF1 [B1]	≥ 2	≥ 5	$O((n-k)n^{d/2} \log n)$	No
	IWFG [B8]	≥ 2	≥ 5	—	Yes
gHSS [D2]	gHSSEx [D5]	2	2	$O(n \log n)$	No
	gHSS2D-u (gHSS2D) [B11]	2	2	$O(nk + n \log n)$	Yes
	gHSS3D-u (gHSS3D) [B11]	3	3	$O(nk + n \log n)$	Yes
	HV4D ⁺ [A10]	4	4	$O(k^3 n)$	Yes*
	WFG-c [A5]	≥ 2	≥ 5	$\Omega(nk^{d/2+1} \log k)$	Yes*
	Chan [A6]	≥ 2	≥ 4	$O(nk^{d/3+1} \text{polylog } k)$	No
LS [D3]/ GSEMO [D4]	FPL [A3]	2	2	$O(tk \log k)$	Yes*
	HV3D ⁺ [A9]	3	3	$O(tk \log k)$	Yes*
	HV4D ⁺ [A10]	4	4	$O(tk^2)$	Yes*
	HBDA-NI [A8]	≥ 2	5, 6	$O(tk^{\lfloor \frac{d}{2} \rfloor})$	Yes*
	WFG [A5]	≥ 2	≥ 7	$\Omega(tk^{d/2} \log k)$	Yes*
	QHV/QHV-II [A7]	≥ 2	≥ 7	$O(t2^{d(k-1)})$	Yes*
	Chan [A6]	≥ 2	≥ 4	$O(tk^{d/3} \text{polylog } k)$	No
EPTAS [D6]		-	≥ 2	$O(n(k + \log n))$	No

combination with the greedy approach but should be easily integrated, have *Availability* marked as “Yes*”. Note that in the case of $d = 2$, there are very efficient algorithms to compute HSSP exactly (see Section 6) and thus, it is questionable whether a greedy algorithm is useful in this case. Anyway, this case is also considered in this summary for completeness.

For the decremental greedy approach, the recommendation is to use the best algorithms to solve the LEASTCONTRIBUTOR problem or, in the absence of one, the UPDATEALLCONTRIBUTIONS or the ALLCONTRIBUTIONS problem: UHVC2D, HVC3D-U, and HVC4D for $d = 2$, $d = 3$, and $d = 4$ cases, respectively. The greedy versions based on the latter two are referred in Reference [49] as gHSSD3D and gHSSD4D, respectively. For the $d > 4$ case, BF1 could be used but since it is not available online, the alternative is to use IWFG. For the incremental greedy approach for $d \leq 3$, the dedicated greedy algorithms are recommended: gHSSEx and gHSS2D for the $d = 2$ case, and gHSS3D for the $d = 3$ case. Note that, gHSSEx is asymptotically faster than gHSS2D only for $k > \log n$, but at the cost of more complex data structures [46]. For the $d = 4$ and $d > 4$ cases, the recommendation is to iteratively solve ONECONTRIBUTION problems with HV4D⁺ and WFG-c (the version of WFG discussed in Section 5.6), respectively, and use the lazy evaluation approach [34] to reduce the number of ONECONTRIBUTION problems to be solved. For the remaining greedy approaches, LS and GSEMO, the recommended algorithms are the fastest ones for HYPERVOLUME problem with available implementations: HV3D⁺ for $d = 3$, HV4D⁺ for $d = 4$, HBDA-NI for $d = 5, 6$, and QHV/QHV-II and WFG for $d \geq 7$.

Currently, only gHSS, gHSSD, and EPTAS provide approximation quality. However, the best one cannot be decided based on these guarantees as these only provide a lower bound and do not imply which algorithm performs better in practice.

Finally, when comparing generic greedy algorithms such as gHSSD, gHSS, LS, and GSEMO, it is important to have in mind that the runtime required by them relies on an algorithm

for a hypervolume-based problem and the choice of such an algorithm does not impact the approximation quality. Thus, it is important that comparisons in terms of approximation quality are (also) conducted independently of the algorithms runtime as better algorithms for hypervolume-based problems may be developed/implemented in the future, which would lead to faster instances of such generic greedy algorithms.

7 CONCLUDING REMARKS

In this article, the properties of the hypervolume indicator were reviewed, namely, strict monotonicity, scaling independence, optimal μ -distributions, and submodularity. These properties tell us what sets the indicator prefers, i.e., which sets the indicator is expected to benefit (closer to the Pareto front and well spread, but depending on the slope of the front). All of these properties reinforce the interest in hypervolume-based selection in EMOAs. Computational cost has been pointed out as its main drawback, but hypervolume-based assessment and selection has become more affordable in the recent years, particularly in low-dimensional cases and/or small problem sizes. Although hypervolume-based selection is still limited for $d \geq 3$, the existence of (fast) greedy algorithms with known approximation ratio are a good alternative.

Regarding algorithmic aspects, the dimension-sweep approach is at the core of faster algorithms tailored for specific low-dimensional cases, but is also effectively used in more general-purpose algorithms when combined with techniques such as bounding. Approaches such as those based on spatial divide-and-conquer and local upper bounds so far have shown to be more competitive in higher-dimensional cases. The exploitation of update algorithms and of the relation between hypervolume-based problems play an important role as well. This is clear with greedy algorithms that greatly benefit from faster algorithms to compute/update hypervolume contributions. The asymptotical behavior of an algorithm, though important, is not the single determinant factor of an algorithm's efficiency. Careful implementations and exploitation of techniques such as bounding and objective reordering, may lead to fast algorithms in practice.

Besides the development of more efficient algorithm and implementations for hypervolume-based problems in general, the greatest challenges still lay in the efficient computation (and even the update) of the HSSP for three and more dimensions. Even though the problem is NP-hard, it is reasonable to aim for efficient algorithms for a low number of dimensions and/or not very large input/output set sizes. This would allow to extend the use of the HSSP in EMOAs for k smaller than $n - 1$, and also to experimentally study exact optimal μ -distributions in more than two dimensions. A second challenge is the development of more efficient algorithms for the computation and, particularly, the update of hypervolume contributions, including the least and greatest contributor, for more than three/four dimensions. This would be beneficial in greedy hypervolume subset selection and for hypervolume-based EMOAs in general, where the iterative process of EMOAs can be exploited to save time.

The hypervolume indicator continues to be widely used, which makes the development of more efficient hypervolume-based algorithms desirable. By discussing the existing techniques, how the hypervolume problems relate, and describing the core ideas of the existing algorithms, this article attempts to help and motivate for further algorithmic development. Moreover, for both benchmarking and faster optimization problem-solving purposes, it is important to make use of the fastest algorithms available. This article provides a rundown of the existing ones, the corresponding links to source code, and recommends which algorithm to use in each case. Hence, this article attempts to be the starting point for the development of new algorithms for hypervolume-related problems, and be a reference for hypervolume-based benchmarking and problem solving.

APPENDICES

A LINKS TO SOURCE CODE

Table 5 summarizes the links³ to source code of implementations of the algorithms described in Sections 4 to 6.

Table 5. List of Links to Source Codes

Algorithm	Link
HSO [A2]	ftp://ftp.tik.ee.ethz.ch/pub/people/zitzler/hypervol.c
FPL [A3]	http://lopez-ibanez.eu/hypervolume
HOY [A4]	ls11-www.cs.tu-dortmund.de/people/beume/publications/hoy.cpp
WFG [A5]	http://www.wfg.csse.uwa.edu.au/hypervolume/
QHV [A7]	http://web.tecnico.ulisboa.pt/luis.russo/QHV/#down
QHV-II [A7]	https://sites.google.com/prod/view/qhv-ii/qhv-ii
HBDA [A8]	https://github.com/renaudlr/hbda
HV3D [A9]	http://lopez-ibanez.eu/hypervolume
HV3D ⁺ [A9]	https://github.com/apguerreiro/HVC
HV4D [A10]	https://doi.org/10.5281/zenodo.285214
HV4D ⁺ [A10]	https://github.com/apguerreiro/HVC
exQHV [B2]	http://web.tecnico.ulisboa.pt/luis.russo/QHV/#down
EF [B3]	https://liacs.leidenuniv.nl/~csmoda/code/hvcon3d.zip
HVC3D [B3]	https://github.com/apguerreiro/HVC
HVC4D [B4]	https://github.com/apguerreiro/HVC
IWFG [B8]	http://www.wfg.csse.uwa.edu.au/hypervolume/
HypSSP [C3]	http://hpi.de/friedrich/docs/code/ssp.zip
extreme2d [C4]	https://gitlri.lri.fr/groz/hssp-hypervolume-contributions
k-LSP [C5]	https://github.com/luis-paquete/2DHSSP
B&B [C8]	https://github.com/rgoomes/hssp
GM-sc [C9]	https://gitlri.lri.fr/groz/hssp-hypervolume-contributions
gHSSD3D [D1]	https://github.com/apguerreiro/HVC
gHSSD4D [D1]	https://github.com/apguerreiro/HVC
gHSS2D [D2]	https://doi.org/10.5281/zenodo.284559
gHSS3D [D2]	https://doi.org/10.5281/zenodo.284559

B DATASETS

To test the performance of the algorithms for hypervolume-related problems, datasets representing different Pareto front shapes are typically used. The following table lists some links where instances of these datasets and scripts to generate them can found online.

The following list includes some of the most used datasets included in the instances and/or scripts available at the provided links, where X , Y , Z represent vectors of random variables, the size of which may differ depending on the dataset. In the wave- w dataset, w is a parameter that indicates the number of convex regions ($w > 0$).

(Spherical) Convex:

$$p_i = 1 - \frac{|X_i|}{\|X\|_2}, \quad X_i \sim \text{Normal}(0, 1), \text{ for } i = 1, \dots, d$$

³In case any of the links becomes inaccessible, the *Wayback Machine* (archive.org) may be used to access it. For example, at the time this article was accepted, the link to the WFG code was down, but it was accessible through the Wayback Machine.

Table 6. List of Links to Datasets and to Scripts to Generate New Datasets

Resource	Link
Instances [85]	http://www.wfg.csse.uwa.edu.au/hypervolume/
Instances [56]	https://www.sites.google.com/view/qhv-ii/qhv-ii
Script and instances [64]	https://github.com/renaudlr/moo-nondominated-sets
Script [72]	http://web.tecnico.ulisboa.pt/luis.russo/QHV/src/generator/generator.tgz
Script [50]	https://github.com/apguerreiro/DataSetGenerator

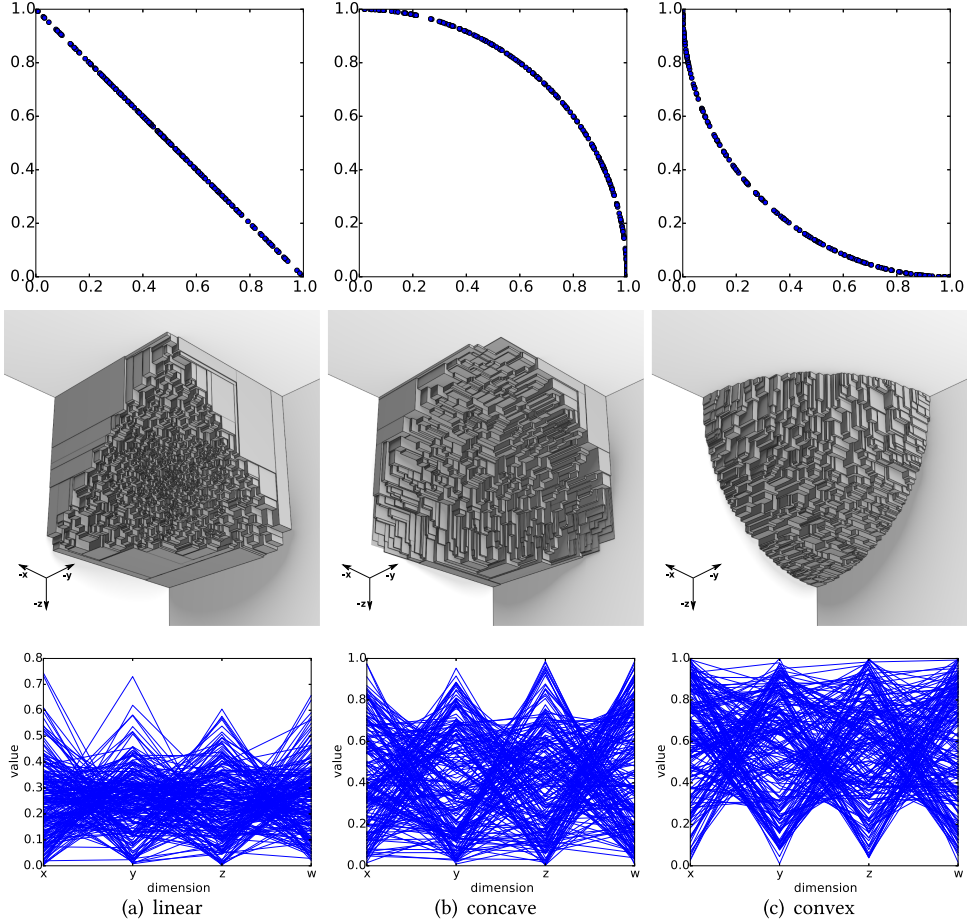


Fig. 14. Datasets: linear (left), concave (middle) and convex (right) in two, three, and four dimensions in the first to third rows, respectively.

(Spherical) Concave:

$$p_i = \frac{|X_i|}{\|X\|_2}, X_i \sim \text{Normal}(0, 1), \text{ for } i = 1, \dots, d$$

Linear:

$$p_i = \frac{|X_i|}{\|X\|_1}, X_i \sim \text{Normal}(0, 1), \text{ for } i = 1, \dots, d$$

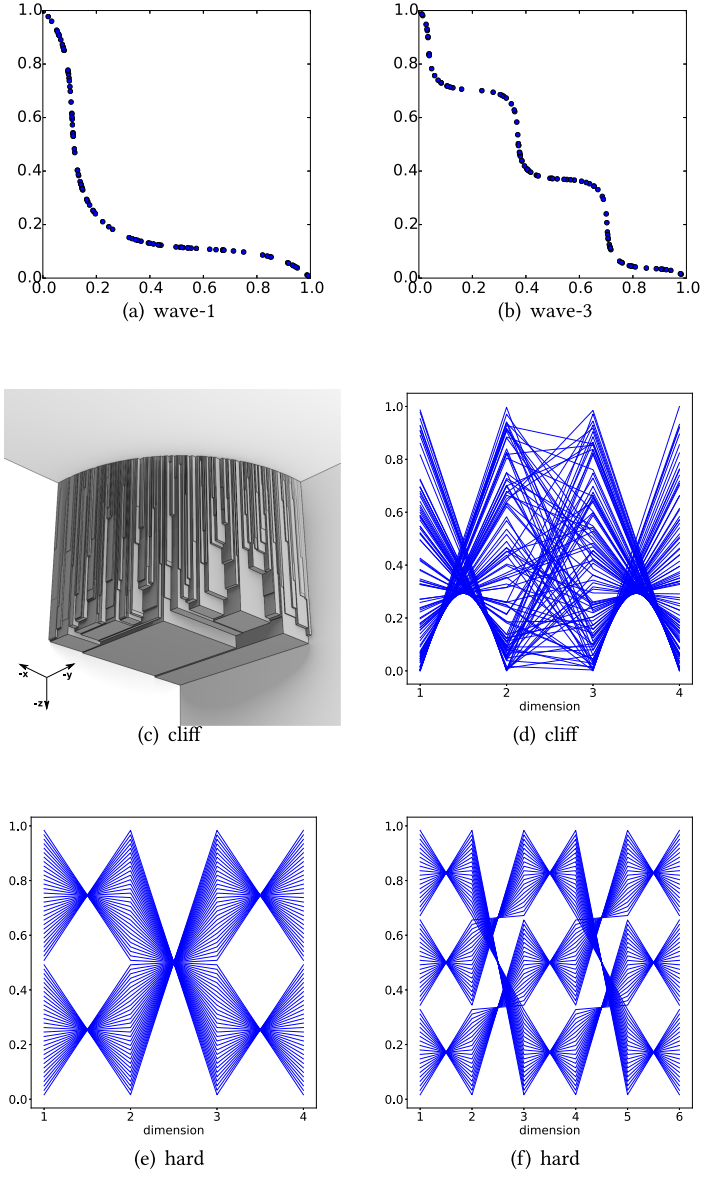


Fig. 15. Datasets: wave-1 and wave-3 in $d = 2$ (top), cliff in $d = 3$ and $d = 4$ (middle), and hard in $d = 4$ and $d = 6$ (bottom).

Wave- w ($d = 2$):

$$\begin{aligned}
 p_1 &= X_1 \cos\left(-\frac{\pi}{4}\right) - Y_1 \sin\left(-\frac{\pi}{4}\right), \quad p_2 = X_1 \sin\left(-\frac{\pi}{4}\right) + Y_1 \cos\left(-\frac{\pi}{4}\right) + 1, \\
 X_1 &= \sqrt{2}Z_1, \quad Y_1 = r \cdot \cos(2w\pi Z_1) - r, \\
 Z_1 &\sim \text{Uniform}(0, 1), \quad r = \frac{0.2}{w}
 \end{aligned}$$

Cliff ($d = 3$):

$$p_i = 1 - \frac{|X_i|}{\|X\|_2}, \quad X_i \sim \text{Normal}(0, 1), \text{ for } i = 1, 2$$

$$p_3 \sim \text{Uniform}(0, 1)$$

Cliff ($d = 4$):

$$p_i = 1 - \frac{|X_i|}{\|X\|_2}, \quad X_i \sim \text{Normal}(0, 1) \text{ for } i = 1, 2$$

$$p_{j+2} = 1 - \frac{|Y_j|}{\|Y\|_2}, \quad Y_j \sim \text{Normal}(0, 1) \text{ for } j = 1, 2$$

Hard ($d = 4$ and n is even):

$$p^j = \left(\frac{n+2j}{2n}, \frac{n-j-1}{n}, \frac{j}{n}, \frac{n-2j-2}{2n} \right) \text{ for } j = 0, \dots, \frac{n}{2} - 1$$

$$p^j = (p_4^l, p_3^l, p_2^l, p_1^l) \text{ for } j = \frac{n}{2}, \dots, n-1 \text{ where } l = j - \frac{n}{2}$$

Some datasets are specific to a given number of dimensions d , which is indicated right after the dataset name. Figures 14 and 15 illustrate the datasets. The linear, convex and concave are the most commonly used datasets [37, 39, 85]. The cliff dataset (in $d = 3, 4$) is a dataset where all points are nondominated in their projection onto the (x, y) -plane [39] (and onto the (z, w) -plane in the $d = 4$ case), and the hard dataset [64] was constructed as a difficult dataset for WFG algorithm (see [A5]). Although the hard dataset as proposed in Reference [64] is for any even number of dimensions, only a simplified version for $d = 4$ is presented here. The wave- w dataset was proposed to test HSSP algorithms on a mixed convex/concave front, and not only on either fully convex or fully concave fronts. Other datasets exist, for example the DTLZ test suit also defines other Pareto fronts that are sometimes used to generate datasets to assess algorithms for hypervolume calculations, such as the degenerate and discontinuous fronts [37]. Another dataset that is more commonly used for the study of the optimal μ -distributions, is the inverted linear dataset [79], which is given by $1 - p_i$ (for $i = 1, \dots, d$) where p is generated as in the linear dataset.

REFERENCES

- [1] Anne Auger, Johannes Bader, and Dimo Brockhoff. 2010. Theoretically investigating optimal μ -distributions for the hypervolume indicator: First results for three objectives. In *Parallel Problem Solving from Nature—PPSN XI*, Robert Schaefer et al. (Eds.). Lecture Notes in Computer Science, Vol. 6238. Springer, Berlin, 586–596.
- [2] Anne Auger, Johannes Bader, Dimo Brockhoff, and Eckart Zitzler. 2009. Investigating and exploiting the bias of the weighted hypervolume to articulate user preferences. In *Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation (GECCO'09)*. ACM, New York, NY, 563–570.
- [3] Anne Auger, Johannes Bader, Dimo Brockhoff, and Eckart Zitzler. 2009. Theory of the hypervolume indicator: Optimal μ -distributions and the choice of the reference point. In *Proceedings of the 10th ACM SIGEVO Workshop on Foundations of Genetic Algorithms (FOGA'09)*. ACM, New York, NY, 87–102.
- [4] Shapour Azarm and Jin Wu. 2001. Metrics for quality assessment of a multiobjective design optimization solution set. *J. Mech. Des.* 123, 1 (2001), 18–25.
- [5] Johannes Bader and Eckart Zitzler. 2011. HypE: An algorithm for fast hypervolume-based many-objective optimization. *Evol. Comput.* 19, 1 (Spring 2011), 45–76.
- [6] Matthieu Basseur, Bilel Derbel, Adrien Goëffon, and Arnaud Liefvooghe. 2016. Experiments on greedy and local search heuristics for d -dimensional hypervolume subset selection. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO'16)*. ACM, New York, NY, 541–548.
- [7] Jon Louis Bentley. 1980. Multidimensional divide-and-conquer. *Commun. ACM* 23, 4 (Apr. 1980), 214–229.
- [8] Nicola Beume. 2009. S-metric calculation by considering dominated hypervolume as Klee's measure problem. *Evol. Comput.* 17, 4 (Winter 2009), 477–492.

- [9] Nicola Beume, Carlos M. Fonseca, Manuel López-Ibáñez, Luís Paquete, and Jan Vahrenhold. 2009. On the complexity of computing the hypervolume indicator. *IEEE Trans. Evol. Comput.* 13, 5 (Oct. 2009), 1075–1082.
- [10] Nicola Beume and Günter Rudolph. 2006. Faster S-metric calculation by considering dominated hypervolume as Klee’s measure problem. In *Proceedings 2nd IASTED Conference on Computational Intelligence* (2006), 231–236.
- [11] Lucas Bradstreet, Luigi Barone, and Lyndon While. 2006. Maximising hypervolume for selection in multi-objective evolutionary algorithms. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC’06)*. 1744–1751.
- [12] Lucas Bradstreet, Luigi Barone, and Lyndon While. 2009. Updating exclusive hypervolume contributions cheaply. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC’09)*. 538–544.
- [13] Lucas Bradstreet, Lyndon While, and Luigi Barone. 2007. Incrementally maximising hypervolume for selection in multi-objective evolutionary algorithms. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC’07)*. 3203–3210.
- [14] Lucas Bradstreet, Lyndon While, and Luigi Barone. 2008. A fast incremental hypervolume algorithm. *IEEE Trans. Evol. Comput.* 12, 6 (Dec. 2008), 714–723.
- [15] Lucas Bradstreet, Lyndon While, and Luigi Barone. 2009. Correction to “A fast incremental hypervolume algorithm.” *IEEE Trans. Evol. Comput.* 13, 5 (Oct. 2009), 1193–1193.
- [16] Lucas Bradstreet, Lyndon While, and Luigi Barone. 2009. A new way of calculating exact exclusive hypervolumes. The University of Western Australia, School of Computer Science & Software Engineering, Technical Report UWACSSSE-09-002 (2009).
- [17] Lucas Bradstreet, Lyndon While, and Luigi Barone. 2010. A fast many-objective hypervolume algorithm using iterated incremental calculations. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC’10)*. 1–8.
- [18] Karl Bringmann. 2010. Klee’s measure problem on fat boxes in time $O(n^{(d+2)/3})$. In *Proceedings of the 26th Annual Symposium on Computational Geometry (SoCG’10)*. ACM, New York, NY, 222–229.
- [19] Karl Bringmann. 2013. Bringing order to special cases of Klee’s measure problem. In *Mathematical Foundations of Computer Science 2013*, Krishnendu Chatterjee and Jiri Sgall (Eds.), Lecture Notes in Computer Science, Vol. 8087. Springer, Berlin, 207–218.
- [20] Karl Bringmann, Sergio Cabello, and Michael T. M. Emmerich. 2017. Maximum volume subset selection for anchored boxes. In *Proceedings of the 33rd International Symposium on Computational Geometry (SoCG’17)*. Dagstuhl Publishing, Schloss Dagstuhl Leibniz-Zentrum für Informatik, Germany, 22:1–22:15.
- [21] Karl Bringmann and Tobias Friedrich. 2008. Approximating the volume of unions and intersections of high-dimensional geometric objects. In *Algorithms and Computation*, Seok-Hee Hong et al. (Eds.), Lecture Notes in Computer Science, Vol. 5369. Springer, Berlin, 436–447.
- [22] Karl Bringmann and Tobias Friedrich. 2009. Approximating the least hypervolume contributor: NP-hard in general, but fast in practice. In *Evolutionary Multi-Criterion Optimization*, Matthias Ehrgott et al. (Eds.), Lecture Notes in Computer Science, Vol. 5467. Springer, Berlin, 6–20.
- [23] Karl Bringmann and Tobias Friedrich. 2009. Don’t be greedy when calculating hypervolume contributions. In *Proceedings of the 10th ACM SIGEVO Workshop on Foundations of Genetic Algorithms (FOGA’09)*. ACM, New York, NY, 103–112.
- [24] Karl Bringmann and Tobias Friedrich. 2010. An efficient algorithm for computing hypervolume contributions. *Evol. Comput.* 18, 3 (Fall 2010), 383–402.
- [25] Karl Bringmann and Tobias Friedrich. 2014. Convergence of hypervolume-based archiving algorithms. *IEEE Trans. Evol. Comput.* 18, 5 (Oct. 2014), 643–657.
- [26] Karl Bringmann, Tobias Friedrich, and Patrick Klitzke. 2014. Two-dimensional subset selection for hypervolume and epsilon-indicator. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation (GECCO’14)*. ACM, New York, NY, 589–596.
- [27] Dimo Brockhoff. 2009. *Many-Objective Optimization and Hypervolume Based Search*. Ph.D. Dissertation. ETH Zurich, Switzerland, Aachen, Germany.
- [28] Dimo Brockhoff. 2010. Optimal μ -distributions for the hypervolume indicator for problems with linear bi-objective fronts: Exact and exhaustive results. In *Simulated Evolution and Learning*, Kalyanmoy Deb et al. (Eds.), Lecture Notes in Computer Science, Vol. 6457. Springer, Berlin, 24–34.
- [29] Dimo Brockhoff, Johannes Bader, Lothar Thiele, and Eckart Zitzler. 2013. Directed multiobjective optimization based on the weighted hypervolume indicator. *J. Multi-Crit. Decis. Anal.* 20, 5 (Jul. 2013), 291–317.
- [30] Dimo Brockhoff, Thanh-Do Tran, and Nikolaus Hansen. 2015. Benchmarking numerical multiobjective optimizers revisited. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation (GECCO’15)*. ACM, New York, NY, 639–646.
- [31] Dimo Brockhoff and Eckart Zitzler. 2007. Improving hypervolume-based multiobjective evolutionary algorithms by using objective reduction methods. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC’07)*. 2086–2093.

- [32] Timothy M. Chan. 2008. A (slightly) faster algorithm for Klee's measure problem. In *Proceedings of the 24th Annual Symposium on Computational Geometry (SoCG'08)*. ACM, New York, NY, 94–100.
- [33] Timothy M. Chan. 2013. Klee's measure problem made easy. In *Proceedings of the 54th IEEE Symposium on Foundations of Computer Science (FOCS'13)*. IEEE Computer Society, Los Alamitos, CA, 410–419.
- [34] W. Chen, H. Ishibuchi, and K. Shang. 2020. Lazy greedy hypervolume subset selection from large candidate solution sets. In *Proceedings of the 2020 IEEE Congress on Evolutionary Computation (CEC'20)*. 1–8.
- [35] W. Cox and L. While. 2016. Improving the IWFG algorithm for calculating incremental hypervolume. In *IEEE Congress on Evolutionary Computation (CEC'16)*. 3969–3976.
- [36] Kerstin Dächert and Kathrin Klamroth. 2015. A linear bound on the number of scalarizations needed to solve discrete tricriteria optimization problems. *J. Global Optim.* 61, 4 (Apr. 2015), 643–676.
- [37] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler. 2002. Scalable multi-objective optimization test problems. In *Proceedings of the 2002 Congress on Evolutionary Computation (CEC'02)*, Vol. 1. 825–830.
- [38] Michael Emmerich, Nicola Beume, and Boris Naujoks. 2005. An EMO algorithm using the hypervolume measure as selection criterion. In *Evolutionary Multi-Criterion Optimization*, Carlos A. Coello Coello et al. (Eds.), Lecture Notes in Computer Science, Vol. 3410. Springer, Berlin, 62–76.
- [39] Michael T.M. Emmerich and Carlos M. Fonseca. 2011. Computing hypervolume contributions in low dimensions: Asymptotically optimal algorithm and complexity results. In *Evolutionary Multi-Criterion Optimization*, Lecture Notes in Computer Science, Vol. 6576. Springer, Berlin, 121–135.
- [40] Michael T. M. Emmerich, André H. Deutz, and Jan Willem Klinkenberg. 2011. Hypervolume-based expected improvement: Monotonicity properties and exact computation. In *Proceedings of the 2011 IEEE Congress of Evolutionary Computation (CEC'11)*. 2147–2154.
- [41] Jonathan E. Fieldsend. 2019. Efficient real-time hypervolume estimation with monotonically reducing error. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO'19)*. ACM, New York, NY, 532–540.
- [42] Mark Fleischer. 2003. The measure of pareto optima applications to multi-objective metaheuristics. In *Evolutionary Multi-Criterion Optimization*, Carlos M. Fonseca et al. (Eds.), Lecture Notes in Computer Science, Vol. 2632. Springer, Berlin, 519–533.
- [43] Carlos M. Fonseca, Luís Paquete, and Manuel López-Ibáñez. 2006. An improved dimension-sweep algorithm for the hypervolume indicator. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC'06)*. 1157–1163.
- [44] Tobias Friedrich and Frank Neumann. 2014. Maximizing submodular functions under matroid constraints by multi-objective evolutionary algorithms. In *Parallel Problem Solving from Nature—PPSN XIII*, Thomas Bartz-Beielstein et al. (Eds.), Lecture Notes in Computer Science, Vol. 8672. Springer International Publishing, 922–931.
- [45] Ricardo J. Gomes, Andreia P. Guerreiro, Tobias Kuhn, and Luís Paquete. 2018. Implicit enumeration strategies for the hypervolume subset selection problem. *Comput. Operat. Res.* 100 (Dec. 2018), 244–253.
- [46] Benoît Groz and Silviu Maniu. 2019. Hypervolume subset selection with small subsets. *Evol. Comput.* 27, 4 (Winter 2019), 611–637.
- [47] Viviane Grunert da Fonseca and Carlos M. Fonseca. 2012. The relationship between the covered fraction, completeness and hypervolume indicators. In *Artificial Evolution*, Jin-Kao Hao et al. (Eds.), Lecture Notes in Computer Science, Vol. 7401. Springer, Berlin, 25–36.
- [48] Andreia P. Guerreiro. 2011. *Efficient Algorithms for the Assessment of Stochastic Multiobjective Optimizers*. Master's thesis. IST, Technical University of Lisbon, Portugal.
- [49] Andreia P. Guerreiro. 2018. *Portfolio Selection in Evolutionary Algorithms*. Ph.D. Dissertation. University of Coimbra, Portugal.
- [50] Andreia P. Guerreiro and Carlos M. Fonseca. 2018. Computing and updating hypervolume contributions in up to four dimensions. *IEEE Trans. Evol. Comput.* 22, 3 (Jun. 2018), 449–463.
- [51] Andreia P. Guerreiro, Carlos M. Fonseca, and Michael T. M. Emmerich. 2012. A fast dimension-sweep algorithm for the hypervolume indicator in four dimensions. In *Proceedings of the Canadian Conference on Computational Geometry (CCCG'12)*. 77–82.
- [52] Andreia P. Guerreiro, Carlos M. Fonseca, and Luís Paquete. 2016. Greedy hypervolume subset selection in low dimensions. *Evol. Comput.* 24, 3 (Fall 2016), 521–544.
- [53] Iris Hupkens and Michael Emmerich. 2013. Logarithmic-time updates in SMS-EMOA and hypervolume-based archiving. In *EVOLVE: A Bridge between Probability, Set Oriented Numerics, and Evolutionary Computation IV*, Michael Emmerich et al. (Eds.), Advances in Intelligent Systems and Computing, Vol. 227. Springer, Heidelberg, 155–169.
- [54] Christian Igel, Nikolaus Hansen, and Stefan Roth. 2007. Covariance matrix adaptation for multi-objective optimization. *Evol. Comput.* 15, 1 (Spring 2007), 1–28.
- [55] Hisao Ishibuchi, Ryo Imada, Yu Setoguchi, and Yusuke Nojima. 2017. Reference point specification in hypervolume calculation for fair comparison and efficient search. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO'17)*. ACM, 585–592.

- [56] Andrzej Jaszekiewicz. 2018. Improved quick hypervolume algorithm. *Comput. Operat. Res.* 90 (2018), 72 – 83.
- [57] A. Jaszekiewicz, R. Susmaga, and P. Zieliński. 2020. Approximate hypervolume calculation with guaranteed or confidence bounds. In *Parallel Problem Solving from Nature—PPSN XVI*, Thomas Bäck et al. (Eds.). Springer International Publishing, Cham, 215–228.
- [58] Haim Kaplan, Natan Rubin, Micha Sharir, and Elad Verbin. 2008. Efficient colored orthogonal range counting. *SIAM J. Comput.* 38, 3 (Jun. 2008), 982–1011.
- [59] Kathrin Klamroth, Renaud Lacour, and Daniel Vanderpooten. 2015. On the representation of the search region in multi-objective optimization. *Eur. J. Oper. Res.* 245, 3 (2015), 767–778.
- [60] Joshua D. Knowles. 2002. *Local-search and Hybrid Evolutionary Algorithms for Pareto Optimization*. Ph.D. Dissertation. Department of Computer Science, University of Reading.
- [61] Joshua D. Knowles, David W. Corne, and Mark Fleischer. 2003. Bounded archiving using the lebesgue measure. In *Proceedings of the 2003 Congress on Evolutionary Computation (CEC’03)*, Vol. 4. 2490–2497.
- [62] Tobias Kuhn. 2015. *Representative Systems and Decision Support for Multicriteria Optimization Problems*. Ph.D. Dissertation. Technische Universität Kaiserslautern.
- [63] Tobias Kuhn, Carlos M. Fonseca, Luís Paquete, Stefan Ruzika, Miguel M. Duarte, and José Rui Figueira. 2016. Hypervolume subset selection in two dimensions: Formulations and algorithms. *Evol. Comput.* 24, 3 (2016), 411–425.
- [64] Renaud Lacour, Kathrin Klamroth, and Carlos M. Fonseca. 2017. A box decomposition algorithm to compute the hypervolume indicator. *Comput. Operat. Res.* 79 (Mar. 2017), 347–360.
- [65] Jussi Laitila and Atte Moilanen. 2017. New Performance guarantees for the greedy maximization of submodular set functions. *Optim. Lett.* 11, 4 (Apr. 2017), 655–665.
- [66] Miqing Li and Xin Yao. 2019. Quality evaluation of solution sets in multiobjective optimisation: A survey. *ACM Comput. Surv.* 52, 2 (Mar. 2019).
- [67] Edgar Manóat López, Luis Miguel Antonio, and Carlos A. Coello Coello. 2015. A GPU-based algorithm for a faster hypervolume contribution computation. In *Evolutionary Multi-Criterion Optimization*, António Gaspar-Cunha et al. (Eds.), Lecture Notes in Computer Science, Vol. 9019. Springer International Publishing, Cham, 80–94.
- [68] Manuel López-Ibáñez, Joshua Knowles, and Marco Laumanns. 2011. On sequential online archiving of objective vectors. In *Proceedings of the International Conference on Evolutionary Multi-Criterion Optimization*, Ricardo H. C. Takahashi et al. (Eds.), Lecture Notes in Computer Science, Vol. 6576. Springer, Berlin, 46–60.
- [69] Laszlo Lovász. 1983. Submodular functions and convexity. In *Mathematical Programming: The State of the Art*, Achim Bachem et al. (Eds.). Springer, Berlin, 235–257.
- [70] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An analysis of approximations for maximizing submodular set functions—I. *Math. Program.* 14, 1 (1978), 265–294.
- [71] Franco P. Preparata and Michael I. Shamos. 1985. *Computational Geometry: An Introduction*. Springer-Verlag, New York, NY.
- [72] Luís M. S. Russo and Alexandre P. Francisco. 2014. Quick hypervolume. *IEEE Trans. Evol. Comput.* 18, 4 (Aug. 2014), 481–502.
- [73] Luís M. S. Russo and Alexandre P. Francisco. 2016. Extending quick hypervolume. *J. Heurist.* 22, 3 (2016), 245–271.
- [74] Johanna Schultes, Michael Stiglmayr, Kathrin Klamroth, and Camilla Hahn. 2021. Hypervolume scalarization for shape optimization to improve reliability and cost of ceramic components. *Optim. Eng.* (2021).
- [75] Britta Schulze, Michael Stiglmayr, Luís Paquete, and Carlos M. Fonseca; David Willems; Stefan Ruzika. 2020. On the rectangular knapsack problem approximation of a specific quadratic knapsack problem. *Math. Methods Oper. Res.* 92 (2020), 107–132.
- [76] Min-Guk Seo and Hyo-Sang Shin. 2018. Greedily excluding algorithm for submodular maximization. In *Proceedings of the 2018 IEEE Conference on Control Technology and Applications (CCTA’18)*. 1680–1685.
- [77] K. Shang, H. Ishibuchi, L. He, and L. M. Pang. 2020. A survey on the hypervolume indicator in evolutionary multi-objective optimization. *IEEE Trans. Evol. Comput.* 25, 1 (2020), 1–20.
- [78] Pradyumn Kumar Shukla, Nadja Doll, and Hartmut Schmeck. 2014. A theoretical analysis of volume based pareto front approximations. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation (GECCO’14)*. ACM, New York, NY, 1415–1422.
- [79] R. Tanabe and H. Ishibuchi. 2020. An Analysis of quality indicators using approximated optimal distributions in a three-dimensional objective space. *IEEE Trans. Evol. Comput.* 24, 5 (2020), 853–867.
- [80] Tamara Ulrich, Johannes Bader, and Eckart Zitzler. 2010. Integrating decision space diversity into hypervolume-based multiobjective search. In *Proceedings of the 12th Annual Conference on Genetic and Evolutionary Computation (GECCO’10)*. ACM, New York, NY, 455–462.
- [81] Tamara Ulrich and Lothar Thiele. 2012. Bounding the Effectiveness of hypervolume-based $(\mu + \lambda)$ -archiving algorithms. In *Learning and Intelligent Optimization*. Lecture Notes in Computer Science, Vol. 7219. Springer, Heidelberg, 235–249.

- [82] Simon Wessing. 2017. Benchmarking the SMS-EMOA with self-adaptation on the bbob-biobj test suite. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO'17)*. ACM, New York, NY, 1772–1779.
- [83] Lyndon While. 2005. A new analysis of the lebmeasure algorithm for calculating hypervolume. In *Evolutionary Multi-Criterion Optimization*, Carlos A. Coello Coello et al. (Eds.), Lecture Notes in Computer Science, Vol. 3410. Springer, Berlin, 326–340.
- [84] Lyndon While and Lucas Bradstreet. 2012. Applying the WFG algorithm to calculate incremental hypervolumes. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC'12)*. 1–8.
- [85] Lyndon While, Lucas Bradstreet, and Luigi Barone. 2012. A fast way of calculating exact hypervolumes. *IEEE Trans. Evol. Comput.* 16, 1 (Feb. 2012), 86–95.
- [86] Lyndon While, Lucas Bradstreet, Luigi Barone, and Phil Hingston. 2005. Heuristics for optimizing the calculation of hypervolume for multi-objective optimization problems. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC'05)*, Vol. 3. IEEE, 2225–2232.
- [87] Lyndon While, Philip Hingston, Luigi Barone, and Simon Huband. 2006. A faster algorithm for calculating hypervolume. *IEEE Trans. Evol. Comput.* 10, 1 (Feb. 2006), 29–38.
- [88] H. Yildiz and S. Suri. 2012. On Klee's measure problem on grounded boxes. In *Proceedings of the 28th Annual Symposium on Computational Geometry (SoCG'12)*. ACM, 111–120.
- [89] Xiuling Zhou, Ning Mao, Wenjuan Li, and Chengyi Sun. 2007. A fast algorithm for computing the contribution of a point to the hypervolume. In *Proceedings of the 3rd International Conference on Natural Computation (ICNC'07)*, Vol. 4. 415–420.
- [90] Eckart Zitzler. 1999. *Evolutionary Algorithms for Multiobjective Optimization: Methods and Applications*. Ph.D. Dissertation. ETH Zurich, Switzerland.
- [91] Eckart Zitzler, Dimo Brockhoff, and Lothar Thiele. 2007. The hypervolume indicator revisited: On the design of pareto-compliant indicators via weighted integration. In *Evolutionary Multi-Criterion Optimization*, Shigeru Obayashi et al. (Eds.), Lecture Notes in Computer Science, Vol. 4403. Springer, Berlin, 862–876.
- [92] Eckart Zitzler, Kalyanmoy Deb, and Lothar Thiele. 2000. Comparison of multiobjective evolutionary algorithms: Empirical results. *Evol. Comput.* 8, 2 (Summer 2000), 173–195.
- [93] Eckart Zitzler, Joshua Knowles, and Lothar Thiele. 2008. Multiobjective optimization. *Quality Assessment of Pareto Set Approximations*. Lecture Notes in Computer Science, Vol. 5252. Springer, Berlin, 373–404.
- [94] Eckart Zitzler and Simon Künzli. 2004. Indicator-based selection in multiobjective search. In *Parallel Problem Solving from Nature—PPSN VIII*, Xin Yao et al. (Eds.), Lecture Notes in Computer Science, Vol. 3242. Springer, Heidelberg, 832–842.
- [95] Eckart Zitzler and Lothar Thiele. 1998. Multiobjective optimization using evolutionary algorithms – A comparative case study. In *Parallel Problem Solving from Nature—PPSN V*, Agoston E. Eiben et al. (Eds.), Lecture Notes in Computer Science, Vol. 1498. Springer, Berlin, 292–301.
- [96] Eckart Zitzler and Lothar Thiele. 1999. Multiobjective evolutionary algorithms: A comparative case study and the strength pareto approach. *IEEE Trans. Evol. Comput.* 3, 4 (Nov. 1999), 257–271.
- [97] Eckart Zitzler, Lothar Thiele, and Johannes Bader. 2010. On set-based multiobjective optimization. *IEEE Trans. Evol. Comput.* 14, 1 (Feb. 2010), 58–79.

Received March 2020; revised January 2020; accepted March 2021